

Universidad Internacional de La Rioja

Escuela Superior de Ingeniería y Tecnología

Grado en Ingeniería Informática

Definición y metodología de trabajo de un equipo de desarrollo de software

Trabajo fin de estudio presentado por:	Felipe Jorge Holgado
Director/a:	Luis Mancera Pascual
Fecha:	29/6/2026
Repositorio del código fuente:	https://github.com/felipejorgeh/Delta.NearAcademy

Resumen

El desarrollo de un proyecto software de cierta envergadura y mantenible en el tiempo es un reto importante. No sólo se requiere aportar una solución de software cuyo desarrollo esté acotado en el tiempo, sino que es necesario implementarla de forma adecuada para facilitar la corrección de errores y su mantenimiento futuro. Además, ante un mercado cambiante, es necesario adaptar y adecuar los desarrollos de forma ágil, modificando la planificación o cambiando el diseño según avanza el desarrollo.

Disponer de un equipo nos permite agilizar el desarrollo, permitiéndonos escalar el número de desarrolladores. Cómo coordinar y alinear el trabajo del equipo es fundamental para controlar el proyecto. Escoger un marco de trabajo adecuado es esencial.

No debemos descuidar la implementación del código, que debe ser dimensionada y supervisada por el equipo. Seguir patrones de desarrollo adecuados y comunes facilita la implementación y mantenibilidad del código. El equipo de desarrollo debe apoyarse en metodologías y herramientas, para desarrollar, testear y desplegar el software.

La definición de estas pautas determinará el éxito de un proyecto.

Palabras clave: SCRUM, CI/CD, Git, Clean Code, DevOps

Abstract

Developing and maintaining a software project of significant size is a major challenge. It is not only necessary to deliver a software solution within a defined timeframe, but also to implement it properly to facilitate error correction and future maintenance. Furthermore, in a constantly changing market, it is essential to adapt developments in an agile manner, adjusting planning or modifying design as development progresses.

Having a team allows development to be accelerated, enabling the number of developers to scale. How to coordinate and align the team's work is fundamental to controlling the project. Choosing an appropriate framework is essential.

The implementation of the code must not be neglected. It must be planned and supervised by the team. Following appropriate and common development patterns facilitates the implementation and maintainability of the code. The development team must rely on methodologies and tools to develop, test and deploy the software.

The definition of these guidelines will determine the success of a project.

Keywords: SCRUM, CI/CD, Git, Clean Code, DevOps

Índice de contenidos

1. Introducción 1
1.1. Motivación 2
1.2. Planteamiento del trabajo 2
1.3. Estructura del trabajo 3
2. Contexto y Estado del Arte 6
2.1. La complejidad del desarrollo de software en equipos profesionales 6
2.1.1. Evolución de los procesos de desarrollo 6
2.1.2. La problemática del equipo como unidad de producción 7
2.2. Estado del arte 8
2.2.1. Metodologías ágiles de gestión del proceso 8
2.2.2. Prácticas técnicas: calidad del código 9
2.2.3. Estrategias de pruebas 10
2.2.4. Gestión del código fuente 11
2.2.5. Integración y entrega continua (CI/CD) 11
3. Objetivos y metodología de trabajo 12
3.1. Objetivo general 12
3.2. Objetivos específicos 12
3.3. Metodología de trabajo 12
4. Definición del marco de trabajo basado en SCRUM 14
4.1. Sprint 14
4.2. Historia de usuario 14
4.3. Backlog 14
4.4. Roles 14
4.4.1. Product Owner 15

4.4.2. Desarrollador 15
4.4.3. Líder 15
4.4.4. Scrum Master 16
4.5. Ceremonias 16
4.5.1. Refinamiento del Backlog 16
4.5.2. Pre Sprint Planning 17
4.5.3. Next Sprint 17
4.5.4. Sprint Planning 17
4.5.5. Daily 18
4.5.6. Pre Sprint Review 18
4.5.7. Sprint Review 19
4.5.8. Retrospectiva 19
4.6. Herramienta de gestión del trabajo 20
5. Patrones y Calidad del Código 22
5.1. Clean code 22
5.1.1. Nombres significativos 23
5.1.2. Funciones pequeñas con responsabilidad única 23
5.1.3. Formato 23
5.1.4. Clases 24
5.1.5. Ejemplo de código legible y mantenible 24
5.2. Principios SOLID 27
5.3. Refactoring 28
5.3.1. Qué es el refactoring 28
5.3.2. Code smells 29
5.3.3. Cuándo aplicar refactoring 30

6. Estructura de proyectos 31
6.1. Organización de proyectos 31
6.1.1. Domain 31
6.1.2. Business 32
6.1.3. Repository 32
6.1.4. Web 32
6.1.5. Injector 32
6.1.6. Test 32
6.2. Organización de clases e interfaces 34
6.2.1. Application 34
6.2.2. Service 34
6.2.3. Repository 34
6.2.4. Controller 34
6.3. Módulo Core 35
6.4. Módulo Startup 36
7. Dimensionamiento y generación de tareas 37
7.1. Liderazgo 37
7.2. Dinámica de reunión 37
7.3. Ejemplo de reunión de dimensionamiento 38
8. Pruebas 41
8.1. Tests de código 41
8.1.1. Tipos de tests 42
8.1.2. Código testeable 43
8.2. Casos de prueba y equipo de QA 44
9. Flujo de trabajo con Git 46

9.1. Ramas feature y task 46
9.2. Ramas release y hotfix 47
10. Integración continua y entrega continua (CI/CD) 50
10.1. Integración continua (CI) 50
10.1.1. Pull request 51
10.2. Entrega continua (CD) 51
11. Plataforma DevOps 53
12. Scrum de Scrums 55
12.1. Principios de Scrum at Scale 55
12.2. Scrum of Scrums 55
12.3. Meta Scrum 56
12.4. Aplicación práctica de Scrum at Scale 56
13. Evaluación de resultados basada en experiencia profesional 58
14. Conclusiones y trabajo futuro 60
14.1. Conclusiones del trabajo 60
14.2. Líneas de trabajo futuro 61
Referencias bibliográficas 63

Índice de figuras

Figura 1 Generar primos, código no legible 25

Figura 2 Generar primos, código legible 26

Figura 3 Estructura de proyectos Delta.NearAcademy 33

Figura 4 Pirámide de testing 43

Figura 5 Ramas feature y task 47

Figura 6 Ramas release 49

1. Introducción

Diseñar, implementar y mantener un proyecto software es un reto importante que debe estar perfectamente definido.

El desarrollo de un proyecto software comprende tres fases fundamentales una vez que los requisitos del sistema están definidos. La fase de diseño traduce esos requisitos en una arquitectura que define la construcción del sistema. La implementación es la fase en la que los desarrolladores materializan el diseño en código. Por último, el mantenimiento abarca todas las actividades posteriores a la entrega: corrección de errores, adaptación a nuevos requisitos y mejora continua del sistema a lo largo de su vida útil.

Un diseño adecuado es fundamental para una correcta implementación. Antes de empezar un desarrollo se debe conocer perfectamente que se quiere implementar. No es necesario, ni recomendable, que todas las características del proyecto estén perfectamente detalladas, pueden existir conceptos sin desarrollar, pero si se desarrolla un ítem concreto debemos conocer los requisitos.

El tiempo de implementación resulta un factor importante que puede condicionarse aumentando el número de integrantes de un equipo, pero la coordinación de los miembros debe ser correcta para que el proyecto progrese adecuadamente.

El mantenimiento del proyecto, mantenimiento del código, es un factor que en muchos proyectos no recibe la importancia requerida y que puede hacer fracasar un proyecto.

Definir un marco de trabajo adecuado para un equipo y dotarlo de metodologías y herramientas se convierte en un objetivo indispensable para la supervivencia de un proyecto.

En este trabajo detallaremos los requisitos necesarios para el funcionamiento de un equipo de desarrollo, atacando aspectos como el marco de trabajo y coordinación, manejo de especificaciones, implementación y pruebas.

1.1. Motivación

En el desarrollo de un proyecto nos podemos encontrar con diversos problemas, pero uno de los más graves es la ausencia de un verdadero equipo de desarrollo. Es habitual encontrar proyectos implementados por desarrolladores totalmente desacoplados entre sí, trabajando como islas: cada uno con su parcela de conocimiento, sin visión global del sistema y sin cultura de colaboración técnica real.

En esta situación extrema es imposible desarrollar ningún marco de trabajo de equipo ni ninguna metodología coordinada. El código crece sin un patrón definido y su legibilidad y mantenimiento se vuelve difícil, comprometiendo la sostenibilidad del proyecto a medio y largo plazo.

1.2. Planteamiento del trabajo

Un proyecto software de un tamaño considerable debe ser gestionado por varios desarrolladores coordinados entre sí, formando un verdadero equipo. Este equipo de desarrollo debe disponer de un marco de trabajo bien definido, que establezca una forma común de organizarse y de tomar decisiones.

Compartir el conocimiento debe ser un pilar básico del equipo. Cuando cualquier persona o figura de la empresa necesite consultar alguna característica del software, debe dirigirse al equipo; es el equipo quien gestiona la petición, no una persona en concreto. Esta visión colectiva del conocimiento elimina la dependencia de individuos y protege la continuidad del proyecto.

Además, el equipo debe apostar por implementar un código mantenible y legible, entendiendo la calidad técnica no como una tarea individual sino como una responsabilidad compartida.

A lo largo de este trabajo exploraremos las metodologías ágiles y las adaptaremos, introduciendo los matices necesarios para aterrizarla en la realidad de un equipo concreto. Mediante estas metodologías dotaremos al equipo de desarrollo de un marco de trabajo que defina su día a día y le dote de mecanismos de planificación y control del desarrollo.

Utilizaremos una herramienta de gestión del trabajo que dé cabida a los conceptos introducidos por la metodología de trabajo. Esta herramienta no funcionará de forma aislada,

sino integrada con otras herramientas necesarias para cubrir todas las necesidades que surgen a lo largo de la vida de un proyecto software.

Prestaremos atención a unos criterios básicos de implementación, introduciendo conceptos de codificación y referencias a obras de referencia sobre buenas prácticas de desarrollo. Estableceremos además una estructura y nomenclatura común para los proyectos de código, como punto de partida sólido para arrancar el desarrollo de cualquier proyecto.

Definiremos el concepto de reunión de dimensión, orientada a planificar un desarrollo y generar las tareas de implementación necesarias. El objetivo es que, antes de que cualquier miembro del equipo escriba una sola línea de código, quede detallado cómo se va a implementar cada tarea, garantizando así una visión técnica compartida desde el inicio.

Introduciremos el concepto de pruebas de código y pruebas de usuario. Las pruebas de código se ejecutan periódicamente en un servidor, bien ante la introducción de nuevo código en el repositorio, bien de manera programada. Las pruebas de usuario se corresponden con una lista de pasos para validar el desarrollo en ejecución; es recomendable contar con un equipo de pruebas (QA) para enriquecer y reforzar este proceso.

Como repositorio de código utilizaremos Git, apoyándonos en una estrategia de ramas para aislar los diferentes desarrollos.

Protegeremos la inserción de modificaciones en el código mediante políticas de confirmación de cambios, conocidas como *pull requests*. Para la validación del código implementaremos una estrategia de integración continua y despliegue continuo, conocida por las siglas CI/CD. Introduciremos asimismo el concepto de versionado del producto, como mecanismo para controlar y comunicar la evolución del software a lo largo del tiempo.

Por último, si el número de desarrolladores crece por necesidades del proyecto, será necesario formar diferentes equipos coordinados entre sí. Para ello introduciremos la metodología SCRUM at Scale, que permite extender los principios ágiles a organizaciones con múltiples equipos trabajando sobre un mismo producto.

1.3. Estructura del trabajo

Después de una breve introducción, en el capítulo 2 expondremos las dificultades existentes a la hora de gestionar un proyecto software. Analizaremos la evolución de las prácticas

existentes y sus problemáticas. Continuaremos analizando las diferentes opciones que disponemos para confeccionar una metodología de trabajo en un equipo de desarrollo.

En el capítulo 3 definiremos los objetivos del trabajo y la metodología seguida para su elaboración, combinando la revisión bibliográfica con la experiencia profesional acumulada a lo largo de veinte años de actividad como desarrollador y líder de equipo.

El capítulo 4 establece el marco de trabajo basado en SCRUM. Se definen los conceptos fundamentales de la metodología, los roles del equipo, las ceremonias que estructuran el trabajo y las herramientas de gestión necesarias para su funcionamiento diario.

El capítulo 5 aborda los patrones y la calidad del código, tomando como referencia las obras *Clean Code* de Robert C. Martin y *Refactoring* (Fowler, 1999). Se detallan los principios de codificación que el equipo debe aplicar de forma consistente para garantizar la legibilidad y mantenibilidad del código a largo plazo.

El capítulo 6 propone una estructura de proyectos, con convenciones de nomenclatura y organización que permiten al equipo orientarse de forma intuitiva en cualquier proyecto que siga el patrón definido.

El capítulo 7 desarrolla el proceso de dimensionamiento y generación de tareas, describiendo la dinámica de las reuniones de dimensionamiento e ilustrando el proceso con un ejemplo concreto basado en un proyecto.

El capítulo 8 aborda las pruebas, distinguiendo entre los tests de código automatizados y los casos de prueba funcionales ejecutados por el equipo de QA.

El capítulo 9 define la estrategia de trabajo con Git, incluyendo el flujo de ramas, y los mecanismos de parches y versionado para gestionar correcciones en versiones desplegadas.

El capítulo 10 describe el proceso de integración continua y entrega continua (CI/CD), introduciendo el concepto de pull requests.

El capítulo 11 presenta la plataforma DevOps como entorno integrado que da soporte a todos los elementos del marco de trabajo: gestión del proceso Scrum, repositorio de código, CI/CD y casos de prueba.

El capítulo 12 introduce la metodología Scrum at Scale, que permite extender los principios ágiles a organizaciones con múltiples equipos trabajando sobre un mismo producto.

Finalmente, el capítulo 13 recoge la evaluación de resultados basada en experiencia profesional y el capítulo 14 las conclusiones del trabajo y las líneas de trabajo futuro.

2. Contexto y Estado del Arte

2.1. La complejidad del desarrollo de software en equipos profesionales

El desarrollo de software, actualmente, combina habilidades técnicas, organización de procesos y coordinación entre desarrolladores. En el entorno profesional actual, los proyectos que se diseñan e implementan son complejos, los plazos son ajustados y la calidad exigente. El equipo de desarrollo de software se convierte en una unidad fundamental de producción dentro de la organización empresarial.

Cómo lograr que un equipo de desarrollo funcione correctamente y plasme las ideas empresariales en una solución se convierte en un reto que no es trivial. Las respuestas han evolucionado desde modelos formales y documentados, como el modelo en cascada, hasta enfoques más adaptativos e iterativos que reconocen la naturaleza cambiante de los requisitos.

No sólo es necesario escoger un marco de trabajo adecuado, se requiere una disciplina o metodología técnica que garantice el mantenimiento del proyecto a medio y largo plazo. El concepto de deuda técnica, coste futuro acumulado por tomar decisiones rápidas o no óptimas, cobra una dimensión vital.

En la mayoría de las situaciones el funcionamiento no óptimo de un equipo de desarrollo, y la gestión no eficiente de un proyecto no se debe únicamente a un factor, pero los citados anteriormente, marco de trabajo y metodología técnica, son sin duda dos indicadores fundamentales para valorar el funcionamiento de un equipo de desarrollo.

2.1.1. Evolución de los procesos de desarrollo

El modelo en cascada fue una de las primeras aproximaciones sistemáticas al desarrollo del software. Su estructura secuencial: análisis, diseño, implementación, pruebas y mantenimiento, definió un marco reproducible, pero resulta rígido frente a requisitos cambiantes.

Winston W. Royce (Royce, 1970) realizó una crítica a este modelo de desarrollo e introdujo una variante fundamental, la iteración (definición de fases repetitivas) y la incrementalidad (el proyecto evoluciona añadiendo características).

En los años noventa apareció otro modelo iterativo e incremental, RUP (*Rational Unified Process)* (Kruchten, 2003). RUP profundiza en el concepto de iteraciones cortas dentro de fases más amplias, acercando el producto a su estado final de forma progresiva.

En 2001 se produjo un punto de inflexión con la aparición del manifiesto Agile (Beck et al., 2001), firmado por un conjunto amplio de profesionales del software. El manifiesto postula cuatro valores fundamentales:

- Individuos e interacciones sobre procesos y herramientas
- Software funcionando sobre documentación extensiva
- Colaboración con el cliente sobre negociación contractual
- Respuesta ante el cambio sobre seguir un plan

2.1.2. La problemática del equipo como unidad de producción

El desarrollo de un proyecto de software en el ámbito profesional es una actividad colectiva coordinada.

Un proyecto de cierta envergadura requiere más de un desarrollador, no solo en términos de volumen y velocidad de trabajo, sino en la diversidad de conocimientos que exige: arquitectura del sistema, lógica de negocio, infraestructura, seguridad, pruebas, experiencia de usuario. Ningún perfil individual puede dominar con solvencia estas cualidades de forma simultánea, y pretender hacerlo compromete tanto la calidad del resultado como la sostenibilidad del proyecto a largo plazo.

El problema se agrava cuando un conjunto de desarrolladores trabaja de forma separada en un proyecto de software. En este escenario, cada desarrollador construye y mantiene su propia parte del sistema sin que el resto del equipo comprenda qué hace ni cómo funciona. Las consecuencias son muy negativas: el código implementado refleja exclusivamente el nivel técnico de quien lo escribe, sin el efecto corrector y de homogenización del equipo; si ese nivel es insuficiente, el resultado es código que funciona en el corto plazo pero que resulta difícilmente mantenible en el futuro. Errores de diseño, ausencia de tests, solapamiento de funcionalidades y nomenclatura arbitraria se consolidan sin que nadie los cuestione.

El trabajo colectivo y conocimiento compartido es, por tanto, una propiedad estructural del equipo y no un objetivo secundario. Cuando el conocimiento sobre el funcionamiento de un módulo o un componente reside exclusivamente en la mente de una persona, el equipo

adquiere dependencias críticas: la baja o la salida de ese miembro paraliza el trabajo en esa área, y la incorporación de nuevos desarrolladores se convierte en un proceso lento y costoso.

2.2. Estado del arte

Detallaremos a continuación las principales soluciones existentes para los diferentes puntos que se desean abordar en este trabajo.

2.2.1. Metodologías ágiles de gestión del proceso

Las metodologías ágiles representan, en la actualidad, el marco de trabajo dominante en la industria del software. Diferentes estadísticas sitúan el uso de estas metodologías en el 70% de las organizaciones[1]. Si bien la expansión de este conjunto de metodologías es bastante reciente y su proceso de implantación no es sencillo, su uso y funcionamiento en muchas empresas no podemos considerarlo como maduro. Destacamos las siguientes metodologías ágiles:

SCRUM es el marco ágil más ampliamente adoptado. Definido por Schwaber y Sutherland en la Scrum Guide (Schwaber & Sutherland, 2020), organiza el trabajo en iteraciones, generalmente de longitud fija, denominadas sprints. En cada sprint el equipo selecciona un subconjunto de tareas y las desarrolla hasta obtener un incremento potencialmente entregable del producto. Los eventos clave y periódicos del marco de trabajo son: Sprint Planning, Daily Scrum, Sprint Review y Retrospectiva. Estos eventos proporcionan ciclos de inspección y adaptación tanto del producto como del proceso. La Scrum Guide define tres roles: el Product Owner, responsable de maximizar el valor del producto; el Scrum Master, responsable de que el proceso funcione correctamente; y el Development Team, responsable de construir el incremento. SCRUM, sin embargo, es un marco intencionalmente incompleto: no prescribe prácticas de ingeniería. Esta ausencia es reconocida por sus propios creadores, que recomiendan complementarlo con prácticas técnicas provenientes de otras disciplinas.

Extreme Programming (XP), propuesto por Kent Beck (Beck, 1999), sí incorpora prácticas técnicas de forma explícita: desarrollo dirigido por pruebas (TDD), programación en parejas,

[1] Siete de cada diez empresas confían en la metodología 'agile': https://www.expansion.com/economia-digital/2018/05/10/5aeb2acf22601d28378b458b.html#:~:text=Business%20Agility%20Corporation%20(BAC)%20es%20una%20asociaci%C3%B3n,en%20el%20mercado%20como%20consecuencia%20de%20la

refactorización continua, integración continua y diseño e implementación simple. Aunque menos extendido que SCRUM en términos de adopción, Extreme Programming ha tenido una influencia determinante en las prácticas técnicas del movimiento ágil.

Kanban (Anderson, 2010) surge del sistema de producción Toyota y aplica sus principios al desarrollo de software. A diferencia de SCRUM, no organiza el trabajo en iteraciones fijas, sino que optimiza el flujo continuo de trabajo mediante la visualización de tareas en un tablero y la limitación y control del trabajo en curso (Work In Progress, WIP). Kanban es especialmente útil en equipos de mantenimiento o soporte, donde los requisitos llegan de forma continua e impredecible.

2.2.2. Prácticas técnicas: calidad del código

Junto con las metodologías de gestión, la comunidad de software ha elaborado un conjunto consolidado de principios y prácticas orientados a la calidad y mantenibilidad interna del código.

Clean Code, libro de Robert C. Martin (Martin, 2008), define un conjunto de principios para escribir código legible, mantenible y libre de duplicación. Martin argumenta que el código se lee muchas más veces de las que se escribe, por lo que la legibilidad es una propiedad de primera clase. Sus principios más destacados incluyen: el uso de nombres significativos, la creación de funciones pequeñas con una única responsabilidad y la eliminación de comentarios que expliquen lo que el código ya expresa por sí mismo.

Los principios **SOLID**, formulados por Robert C. Martin y resumidos por Michael Feathers bajo el acrónimo, proporcionan cinco principios de diseño orientado a objetos: Responsabilidad Única (Single Responsibility Principle), Abierto/Cerrado (Open/Closed Principle), Sustitución de Liskov (Liskov Substitution Principle), Segregación de Interfaces (Interface Segregation Principle) e Inversión de Dependencias (Dependency Inversion Principle).

En el libro **Refactoring** (Fowler, 1999), se describe el proceso de restructurar el código existente sin modificar su comportamiento. Fowler cataloga más de setenta técnicas de refactorización —desde renombrado de variables hasta extracción de métodos y clases— y argumenta que la refactorización no es una actividad puntual, sino una práctica continua e integrada en el flujo diario de desarrollo. La refactorización continua es, por tanto, el

mecanismo técnico para mantener el código en condiciones de ser extendido y modificado con seguridad.

El concepto de **deuda técnica** describe el coste implícito de las decisiones de diseño deficientes o apresuradas. Al igual que la deuda financiera, la deuda técnica genera intereses: el trabajo adicional que será necesario realizar en el futuro si no se corrige. Su reflejo en el código puede ser nefasto, incrementando de manera exponencial el tiempo de desarrollo o incluso impidiendo la posibilidad de realizar nuevos desarrollos.

2.2.3. Estrategias de pruebas

Las pruebas del software son un área con una larga tradición en la ingeniería del software que ha evolucionado significativamente en el contexto ágil.

En la **pirámide de testing** (Fowler, 2012) se establece una jerarquía de tipos de pruebas. Existen diversos estilos de pirámides dependiendo de la naturaleza del proyecto. Un ejemplo de pirámide: en la base, las pruebas unitarias, rápidas y abundantes; en el nivel intermedio, las pruebas de integración; y en la parte más alta, las pruebas de extremo a extremo (end-to-end), más lentas y costosas. La pirámide, además del volumen de pruebas refleja el coste en tiempo y esfuerzo de las pruebas, cuanto más arriba en la pirámide más costoso.

El Desarrollo Dirigido por Pruebas (**Test-Driven Development, TDD**) (Beck, 2002) invierte el orden convencional: primero se escriben las pruebas. Este ciclo garantiza que el código producido tiene cobertura de pruebas desde el primer momento, y fuerza un diseño más desacoplado, ya que el código debe ser testeable desde el principio.

El Desarrollo Dirigido por el Comportamiento (**Behavior-Driven Development, BDD**) (Smart, 2014) extiende TDD incorporando el lenguaje del dominio de negocio en la definición de las pruebas.

La integración del equipo de **garantía de calidad (QA)** en el proceso de desarrollo ha evolucionado desde un rol de verificación al final de la implementación hasta una participación desde las fases iniciales de desarrollo. Este enfoque traslada la detección de defectos lo antes posible en el proceso, reduciendo el coste de su corrección.

2.2.4. Gestión del código fuente

Git (Chacon & Straub, 2014), creado por Linus Torvalds en 2005, es el sistema de control de código distribuido dominante en la industria. Sobre Git se han construido modelos de flujo de trabajo como Git Flow (Driessen, 2010), que define ramas específicas para el desarrollo de funcionalidades, la preparación de versiones y la corrección de errores, y Trunk Based Development (Hammant, 2020), que propone integrar directamente en la rama principal con mayor frecuencia.

2.2.5. Integración y entrega continua (CI/CD)

La Integración Continua (**Continuous Integration, CI**) (Fowler, 2006) consiste en el proceso de compilación y ejecución de pruebas automática al integrar modificaciones de código en un repositorio. Esta técnica nos garantiza un conocimiento y control del código completos.

Este proceso se refuerza con la definición de políticas de inserción de código en el repositorio. Comúnmente conocidas como *pull request*, este mecanismo nos permite condicionar las modificaciones de código en el repositorio a que el código sea revisado o aceptado por más miembros del equipo, el código compile y las pruebas se ejecuten satisfactoriamente.

La Entrega Continua (**Continuous Delivery, CD**) (Humble & Farley, 2010), extiende la integración continua garantizando que el software siempre se encuentra en un estado desplegable en producción.

3. Objetivos y metodología de trabajo

3.1. Objetivo general

El objetivo general de este trabajo es diseñar un marco de trabajo para equipos de desarrollo de software, que integre metodologías ágiles, prácticas técnicas de calidad y herramientas de gestión, con el fin de dotar a cualquier equipo de los mecanismos necesarios para organizarse, coordinarse y entregar software de forma sostenible y mantenible a lo largo del tiempo.

3.2. Objetivos específicos

Para alcanzar el objetivo general, se plantean los siguientes objetivos específicos:

- Utilizar una metodología ágil para dotar al equipo de desarrollo de un marco de trabajo
- Proponer criterios y buenas prácticas de implementación que aseguren la legibilidad y mantenibilidad del código a medio y largo plazo
- Definir una estructura organizativa del equipo que fomente el conocimiento compartido y elimine las dependencias críticas de personas individuales
- Establecer un proceso de planificación del desarrollo que garantice que cada tarea queda técnicamente detallada antes de su implementación
- Definir una estrategia de gestión del código fuente basada en Git, incluyendo una política de ramas y de confirmación de cambios mediante pull requests
- Integrar un sistema de pruebas de código y pruebas de usuario en el flujo de trabajo del equipo, apoyado en una infraestructura de integración y entrega continua (CI/CD)
- Explorar la escalabilidad del marco propuesto hacia organizaciones con múltiples equipos, mediante la metodología Scrum at Scale

3.3. Metodología de trabajo

Para la elaboración de este trabajo se ha seguido un método de trabajo mixto que combina la revisión bibliográfica con la experiencia profesional acumulada a lo largo de veinte años de actividad como desarrollador y líder de equipo.

La revisión bibliográfica ha permitido fundamentar y contrastar las propuestas presentadas en el estado del arte. Se han consultado obras de referencia de autores reconocidos en la

disciplina, así como guías y manifiestos de amplia adopción en la industria del desarrollo de software.

La experiencia profesional propia constituye la segunda fuente de conocimiento de este trabajo. Las propuestas, decisiones de diseño y recomendaciones que se presentan a lo largo de los siguientes capítulos no son únicamente construcciones teóricas, sino que han sido validadas en entornos reales de desarrollo, con equipos de diferentes tamaños y en proyectos de diversa naturaleza y complejidad. Esta perspectiva práctica permite aterrizar los conceptos teóricos en situaciones concretas, identificar los problemas habituales en la implantación de metodologías y proponer soluciones ajustadas a la realidad del trabajo diario.

4. Definición del marco de trabajo basado en SCRUM

El marco de trabajo propuesto para la gestión de un equipo de desarrollo es SCRUM.

Nos basamos en una metodología SCRUM adaptada con matices a una situación de trabajo.

SCRUM es un marco ágil iterativo e incremental que organiza el trabajo en ciclos cortos de desarrollo, permitiendo al equipo entregar valor de forma continua y adaptarse a los cambios de requisitos sin comprometer la estabilidad del proyecto. Su adopción generalizada en la industria del software lo convierte en un punto de partida sólido y reconocido, sobre el que es posible construir un marco de trabajo propio sin perder el rigor metodológico.

La Guía de Scrum es el documento oficial (Schwaber & Sutherland, 2020), donde se detalla un marco intencionalmente minimalista que debe ser completado con prácticas propias.

4.1. Sprint

Trabajamos en iteraciones de tiempo denominadas sprints. El tiempo de sprint aconsejado es de 2 semanas, se considera un tiempo razonable en el que el equipo puede realizar un trabajo cuantificable y a su vez es un intervalo de tiempo ajustado para aportar nuevas funcionalidades al producto.

En un sprint el equipo de desarrollo implementa historias de usuario.

4.2. Historia de usuario

Una historia de usuario describe una funcionalidad del producto desde la perspectiva del usuario, acompañada de los requisitos necesarios para su implementación.

Las historias de usuario antes de entrar en un sprint existen y están definidas en el backlog.

4.3. Backlog

El backlog es una lista ordenada de historias de usuario, mantenida y priorizada por el Product Owner y estimada en esfuerzo por el equipo. Las historias de usuario en la parte superior del backlog son las más prioritarias y las candidatas a entrar en el siguiente sprint.

4.4. Roles

El tamaño recomendado del equipo oscila entre tres y siete desarrolladores. Detallamos los siguientes roles existentes en un equipo de SCRUM.

4.4.1. Product Owner

Es el responsable de maximizar el valor del producto. Es un perfil de negocio y no pertenece al equipo técnico, no es un desarrollador. Conoce en profundidad las necesidades del cliente y de la organización. Su responsabilidad principal es gestionar y priorizar el backlog del producto, asegurando que el equipo trabaja siempre en aquello que aporta mayor valor. Es el interlocutor principal con el equipo de desarrollo.

4.4.2. Desarrollador

Es el miembro del equipo responsable de implementar las historias de usuario comprometidas en cada sprint. El equipo de desarrollo es autónomo en la ejecución de las tareas y comparte la responsabilidad del resultado: no existen parcelas individuales de conocimiento ni propietarios exclusivos de módulos o componentes. Cualquier miembro del equipo debe poder trabajar en cualquier parte del sistema, aunque puedan existir áreas de mayor especialización individual. Esta propiedad colectiva del código es una condición necesaria para la sostenibilidad del proyecto y para eliminar las dependencias críticas de personas concretas.

4.4.3. Líder

Es el líder técnico y referente del equipo de desarrollo. Es un perfil senior que combina la capacidad técnica con la visión global del sistema. Define la arquitectura, toma las decisiones de diseño relevantes, guía al equipo en la resolución de problemas complejos y vela por la calidad y mantenibilidad del código. Lo ideal es que el líder de equipo no sea un único desarrollador sino un conjunto de desarrolladores senior, idealmente 3 desarrolladores para un equipo de más de 5 personas.

El líder forma parte del equipo de desarrollo y participa activamente en la implementación: es el primer desarrollador del equipo, no una figura exclusivamente gestora. Actúa además como puente entre el equipo y el Product Owner, traduciendo los requisitos de negocio en decisiones técnicas y comunicando las restricciones y posibilidades del sistema en términos comprensibles para el negocio.

En equipos de desarrollo reales, la ausencia de una figura técnica de referencia genera decisiones de arquitectura inconsistentes, deuda técnica acumulada y dificultad para mantener una visión coherente del sistema a lo largo del tiempo.

4.4.4. Scrum Master

Es el responsable de que el proceso funcione correctamente. Facilita las ceremonias del equipo, elimina los impedimentos que puedan bloquear el avance del desarrollo y vela por que los principios ágiles se apliquen de forma coherente. En este marco de trabajo, el Scrum Master no es una figura dedicada en exclusiva a este rol, sino un líder de equipo que lo asume de forma rotatoria en cada sprint.

4.5.Ceremonias

Las ceremonias son los eventos periódicos que estructuran el trabajo del equipo a lo largo del sprint. SCRUM define cuatro ceremonias fundamentales: Sprint Planning, Daily Scrum, Sprint Review y Retrospectiva. A estas reuniones añadimos el Pre Sprint Planning, Refinamiento del Backlog y Pre Sprint Review. Todas ellas tienen un propósito concreto y una duración acotada; celebrarlas con regularidad y disciplina es una condición necesaria para que el marco de trabajo funcione correctamente.

4.5.1. Refinamiento del Backlog

El refinamiento del backlog es una ceremonia periódica en la que el Product Owner presenta nuevas historias de usuario al equipo técnico. El equipo las analiza, realiza las preguntas necesarias para comprenderlas en profundidad y las estima en story points.

Los story points son una unidad de medida relativa del esfuerzo de implementación de una historia de usuario. No representan horas de trabajo, sino una valoración colectiva del equipo que tiene en cuenta la complejidad técnica, la incertidumbre y el volumen de trabajo estimado. La estimación se realiza de forma colectiva, habitualmente mediante la técnica de planning poker, en la que cada miembro del equipo propone su estimación de forma simultánea para evitar que la opinión de unos condicione la de otros.

Una historia de usuario que no ha sido estimada en el refinamiento del backlog no puede entrar en un sprint. Esta condición es fundamental: garantiza que el equipo ha comprendido y analizado la historia antes de comprometerse a implementarla, y evita que entren en el sprint historias con requisitos ambiguos o incompletos que generen bloqueos durante el desarrollo.

4.5.2. Pre Sprint Planning

El Pre Sprint Planning se celebra el penúltimo día del sprint en curso. En ella participan el Product Owner y los líderes del equipo. Su objetivo es acordar qué historias de usuario del backlog entrarán en el siguiente sprint.

También se repasa si alguna historia de usuario del sprint en curso no va a terminarse. Si se produce esta situación, puede ocurrir que la historia de usuario pase al siguiente sprint, o por el contrario, que vuelva al backlog.

El interés del Product Owner por terminar un desarrollo concreto puede chocar con la necesidad del equipo de desarrollo de terminar un desarrollo en curso o abordar un refactoring.

Recordamos que sólo las historias de usuario estimadas en story point pueden entrar en un sprint, y por lo tanto ser tratada en esta reunión. Esta condición garantiza que el equipo ha comprendido y analizado cada historia antes de comprometerse a implementarla.

4.5.3. Next Sprint

La reunión Next Sprint se celebra tras el Pre Sprint Planning, una vez cerrada la lista de historias de usuario que entrarán en el siguiente sprint. Participan exclusivamente los líderes del equipo.

Su objetivo es repasar las historias de usuario a acometer en el siguiente sprint y repartir el liderazgo del dimensionamiento: cada historia de usuario queda asignada a un desarrollador, que será el encargado de dirigir su dimensionamiento durante el Sprint Planning. Esta preparación previa garantiza que el Sprint Planning se desarrolla de forma ordenada y sin improvisación, y que cada historia llega con un responsable técnico que la conoce y ha reflexionado sobre cómo abordarla.

4.5.4. Sprint Planning

El Sprint Planning es el primer evento del sprint. Las historias de usuario que entran en el sprint ya han sido acordadas en la Pre Sprint Planning, por lo que el objetivo de esta reunión no es decidir qué se va a desarrollar sino planificar cómo se va a desarrollar.

El scrum master ha organizado la reunión, estableciendo un orden lógico para abordar el dimensionamiento de las historias de usuario.

El encargado de la dimensión de la historia de usuario lidera el proceso y se identifican y crean las tareas de implementación necesarias para completarla. Este proceso garantiza que, antes de que cualquier miembro del equipo escriba una sola línea de código, existe una visión técnica compartida de cómo se va a afrontar cada desarrollo.

Durante el dimensionamiento de una historia de usuario el equipo al completo participa. Puede ocurrir que el encargado exponga el desarrollo, que todo el equipo lo entienda y acepte, y que las tareas se generen sin mayor discusión. En cualquier caso, una vez generadas las tareas, es necesario que todos los desarrolladores las comprendan.

Si durante el dimensionamiento se detecta que una historia no está suficientemente detallada o existen dudas que no pueden resolverse en el momento, la historia debe ser devuelta al backlog y no comprometida en el sprint.

Sobre el proceso de dimensionamiento volveremos en el capítulo 7.

4.5.5. Daily

El Daily es una reunión diaria de duración máxima de quince minutos que se celebra al inicio de la jornada. Participan exclusivamente los desarrolladores del equipo.

Cada miembro del equipo responde brevemente a tres preguntas: qué hizo el día anterior, qué va a hacer hoy y si existe algún impedimento que bloquee su avance. El objetivo de la reunión es sincronizar al equipo y detectar bloqueos de forma temprana.

Es importante que el Daily no derive en una reunión de resolución de problemas. Los posibles problemas se identifican en ella, pero se resuelven fuera, en conversaciones específicas con las personas implicadas. Si un tema requiere discusión, se anota y se aborda al terminar la reunión entre quienes corresponda, sin retener al resto del equipo.

Es importante no exceder el tiempo de la reunión, en caso de que la reunión se esté dilatando el scrum master debe intervenir y reconducir la reunión.

4.5.6. Pre Sprint Review

La Pre Sprint Review se celebra el penúltimo día del sprint, víspera del Sprint Review. Participan exclusivamente los miembros del equipo de desarrollo. Su objetivo es organizar la presentación de las historias de usuario terminadas durante el sprint.

El equipo se reparte las historias de usuario completadas durante el sprint, de modo que cada desarrollador presenta aquello en lo que ha trabajado. Esta preparación garantiza que la Sprint Review se desarrolla de forma ordenada y fluida, y que todas las historias implementadas quedan correctamente explicadas ante el Product Owner y las partes interesadas.

Es importante no exceder el tiempo de esta reunión. Si la reunión se dilata, el Scrum Master debe intervenir y reconducirla.

4.5.7. Sprint Review

El Sprint Review se celebra el último día del sprint. En la reunión participan el equipo técnico, el Product Owner y otras partes interesadas de la organización. Su objetivo es presentar el incremento desarrollado durante el sprint y recoger opinión.

Cada desarrollador presenta las historias de usuario que le fueron asignadas en el Pre Sprint Review. El Product Owner y las partes interesadas valoran el incremento, formulan preguntas y aportan feedback. Esta opinión puede generar nuevas historias de usuario, modificar historias existentes en el backlog o reordenar sus prioridades.

El Sprint Review no es una reunión de validación interna del equipo sino un evento de inspección del producto orientado al negocio. Su valor reside en el contacto directo entre quienes construyen el producto y quienes lo utilizan o financian.

4.5.8. Retrospectiva

La Retrospectiva se celebra el último día del sprint y es conveniente que sea el último evento de la jornada de trabajo. Es una reunión exclusivamente interna del equipo de desarrollo, orientada a la mejora continua del proceso.

El equipo reflexiona sobre tres aspectos del sprint que acaba de concluir: qué ha funcionado bien y debe mantenerse, qué ha funcionado mal y debe corregirse, y qué acciones concretas se van a tomar para mejorar en el siguiente sprint.

La dinámica de la reunión sigue un proceso estructurado. Cada desarrollador dispone de un tiempo de reflexión individual para escribir notas sobre qué ha funcionado bien durante el sprint. Posteriormente el equipo lee todas las notas en conjunto y se abre un turno de debate. El mismo proceso se repite para lo que ha funcionado mal o debe mejorar.

Una vez expuestas todas las notas, el equipo las vota y selecciona las dos más votadas en cada categoría. Sobre estas se vuelve a reflexionar individualmente y se escriben notas con acciones concretas para mejorar lo expuesto. Estas acciones se presentan, se votan y las dos más votadas se convierten en tareas para el siguiente sprint, con un límite de horas de ejecución asignado.

Para incentivar la participación y crear un clima cómodo en el equipo, es buena práctica comenzar la Retrospectiva con una presentación breve: un desarrollador, por turno rotatorio, expone durante diez minutos un tema de su elección, relacionado mínimamente con el desarrollo de software. Esta práctica rompe la rutina, fomenta el aprendizaje compartido y genera un ambiente distendido antes de entrar en la reflexión del sprint.

Es fundamental que la Retrospectiva no genere tensiones ni discusiones entre miembros del equipo. El Scrum Master es el responsable de mantener un clima de respeto y constructividad, reconduciendo la reunión si en algún momento deriva hacia el reproche personal. El objetivo es mejorar el proceso, no señalar a personas.

4.6. Herramienta de gestión del trabajo

El marco de trabajo definido en este capítulo requiere una herramienta de gestión que dé soporte digital a todos sus elementos: backlog, sprints, historias de usuario, tareas y tablero de seguimiento. Una herramienta adecuada no solo almacena la información, sino que la hace visible y accesible para todo el equipo en tiempo real, convirtiéndose en el punto de referencia común del trabajo diario.

La herramienta debe cubrir las siguientes necesidades:

- Gestión del backlog*:*

Permitir crear, priorizar y estimar historias de usuario en story points, manteniendo una lista ordenada y accesible para el Product Owner y el equipo.

- Gestión de sprints:

Permitir crear sprints, asignarles historias de usuario y hacer seguimiento del progreso a lo largo del sprint, incluyendo métricas como el *burndown chart*.

- Tablero kanban:

Ofrecer una vista visual del estado de las tareas del sprint en curso, con columnas configurables que reflejen el flujo de trabajo del equipo.

- Gestión de tareas:

Permitir descomponer cada historia de usuario en tareas técnicas, asignarlas a desarrolladores y registrar la estimación en horas de cada una.

- Trazabilidad:

Mantener la relación entre historias de usuario y tareas, permitiendo navegar desde cualquier tarea hasta la historia que la originó y viceversa.

Es muy interesante que la herramienta se integre además de forma natural con el repositorio de código, permitiendo vincular commits y pull requests a las tareas correspondientes, lo que proporciona trazabilidad completa entre el trabajo planificado y el código implementado.

5. Patrones y Calidad del Código

La codificación por parte de un equipo que trabaja de forma coordinada tiende a homogeneizarse. Es un síntoma positivo para la salud de un equipo que cuando un desarrollador lea el código de otro desarrollador piense que él lo hubiera escrito de igual manera. Esto ocurre cuando se ejerce un liderazgo adecuado y el equipo adquiere automatismos a base de reuniones de trabajo, pero este liderazgo se ha de sustentar en argumentos sólidos sobre codificación. Si en una reunión de dimensionamiento se plantea un debate sobre cómo implementar una pieza de código, podemos resolverlo recurriendo a argumentos establecidos y acordados por todos los desarrolladores como el decálogo de buenas prácticas a seguir.

Una de las obras más relevante, a mi entender, sobre buenas prácticas de implementación es el libro Clean Code (Martin, 2008).

Robert C. Martin es autor de los principios SOLID, una serie de reglas de diseño de software.

Comentaremos en esta sección detalles de otro libro de referencia como es Refactoring (Fowler, 1999).

Para concluir plantearemos una estructura de proyectos y código para profundizar en conceptos y detalles de implementación.

5.1.Clean code

El libro Clean Code (Martin, 2008) parte de una premisa básica y sencilla, el código se lee muchas más veces de las que se escribe. Este aspecto hace que la legibilidad del código sea una característica fundamental de un código.

Ocurre en proyectos con bastante tiempo de vida y un código poco mantenible, código espagueti, que necesitamos ver cómo se comporta una funcionalidad y recurrimos a leer el código. La labor se convierte en un suplicio, el código no permite abstraernos y debemos leer todos los métodos desde el front end hasta el backned. El proceso se convierte en tedioso, debemos ir cargando en la cabeza todo el código, los nombres de los métodos no expresan lo que hacen, existen paso de demasiados parámetros…. Una labor de minutos en un código legible puede convertirse en un trabajo de días.

Detallemos aspectos destacables que hacen a un código legible y destacable:

5.1.1. Nombres significativos

Los nombres de variables, funciones, clases y módulos deben revelar su intención. Un nombre significativo elimina la necesidad de un comentario explicativo y hace que el código se lea como una descripción del problema que resuelve. Además, evita tener que introducirnos en su implementación, sólo con leer el nombre sabemos qué hace.

Un nombre ambiguo o genérico obliga al lector a inferir el significado por el contexto, aumentando el esfuerzo de comprensión y el riesgo de interpretación errónea.

Nombres como data, value, temp o aux no comunican ninguna intención y deben evitarse. Por el contrario, nombres como calculateInvoiceTotal, activeUserList o sendEmailNotification comunican su propósito de forma inmediata. Un buen nombre es aquel que hace innecesaria cualquier explicación adicional.

5.1.2. Funciones pequeñas con responsabilidad única

Una función debe hacer una sola cosa, hacerla bien y no hacer nada más. Una función que hace múltiples cosas es difícil de nombrar, de entender, de probar y de reutilizar. Cuando una función necesita un comentario para explicar qué hace cada bloque interno, es una señal clara de que debe ser dividida en funciones más pequeñas.

El tamaño de una función es un indicador directo de su complejidad. En la práctica, las funciones deben ser lo suficientemente pequeñas como para que su propósito sea evidente a primera vista y su nombre la describa completamente.

5.1.3. Formato

El formato del código es una forma de expresión y comunicación. Un código con un formato coherente y consistente es más fácil de leer y de navegar.

El equipo debe acordar y respetar unas convenciones de formato comunes: tabulación, longitud máxima de línea, espaciado entre bloques y organización de los elementos dentro de una clase o módulo.

El formato no debe dejarse a criterio individual sino automatizarse mediante herramientas de análisis estático y formateo automático integradas en el flujo de desarrollo. De este modo el equipo elimina las discusiones sobre estilo y garantiza la consistencia del código sin esfuerzo manual.

5.1.4. Clases

Las clases deben ser pequeñas, al igual que las funciones. Sin embargo, el tamaño de una clase no se mide en líneas sino en responsabilidades. Una clase debe tener una única razón para cambiar, principio que coincide con el primero de los principios SOLID: el principio de responsabilidad única.

Una clase con demasiadas responsabilidades es difícil de entender, de probar y de mantener. Cuando una clase crece sin control, acumulando métodos y atributos que no guardan relación entre sí, es una señal clara de que debe ser dividida. El nombre de la clase debe describir con precisión su responsabilidad; si para nombrarla necesitamos recurrir a palabras como Manager, Processor o Handler, es probable que esté asumiendo demasiadas responsabilidades.

5.1.5. Ejemplo de código legible y mantenible

Robert C. Martin utiliza en Clean Code el ejemplo de un generador de números primos para ilustrar el proceso de transformación de un código difícil de leer en una pieza limpia y mantenible. El punto de partida es el código de la Figura 1, que genera números primos mediante la criba de Eratóstenes, pero resulta difícil de comprender.

Aunque este código funciona correctamente, presenta numerosos problemas de legibilidad. Los nombres de variables como s, f, i o j no revelan su intención y obligan al lector a deducir su propósito por el contexto. Toda la lógica está concentrada en un único método extenso que asume varias responsabilidades simultáneamente: inicializar las estructuras, cruzar los múltiplos, contar los primos y construir el resultado. La anidación de bucles y condicionales dificulta seguir el flujo del algoritmo, y comprender qué hace cada bloque requiere un esfuerzo considerable.

Partiendo de este código, Robert C. Martin nos ofrece el código de la Figura 2, dónde extrae cada responsabilidad a una función pequeña con un nombre significativo, y obtiene una versión del algoritmo que se lee de forma clara y donde cada función tiene una única responsabilidad.

Figura 1 *Generar primos, código no legible*

```java
public class GeneratePrimes
{
    public static int[] generatePrimes(int maxValue)
    {
        if (maxValue >= 2)
        {
            int s = maxValue + 1;
            boolean[] f = new boolean[s];
            int i;
            for (i = 0; i < s; i++)
                f[i] = true;

            f[0] = f[1] = false;

            int j;
            for (i = 2; i < Math.sqrt(s) + 1; i++)
            {
                if (f[i])
                {
                    for (j = 2 * i; j < s; j += i)
                        f[j] = false;
                }
            }

            int count = 0;
            for (i = 0; i < s; i++)
            {
                if (f[i])
                    count++;
            }

            int[] primes = new int[count];

            for (i = 0, j = 0; i < s; i++)
            {
                if (f[i])
                    primes[j++] = i;
            }

            return primes;
        }
        else
            return new int[0];
    }
}
```

Fuente: Martin, R. C. (2008). Clean Code

Figura 2 *Generar primos, código legible*

```java
public class PrimeGenerator
{
    private static boolean[] crossedOut;
    private static int[] result;

    public static int[] generatePrimes(int maxValue)
    {
        if (maxValue < 2)
            return new int[0];

        uncrossIntegersUpTo(maxValue);
        crossOutMultiples();
        putUncrossedIntegersIntoResult();
        return result;
    }

    private static void uncrossIntegersUpTo(int maxValue)
    {
        crossedOut = new boolean[maxValue + 1];
        for (int i = 2; i < crossedOut.length; i++)
            crossedOut[i] = false;
    }

    private static void crossOutMultiples()
    {
        int limit = determineIterationLimit();
        for (int i = 2; i <= limit; i++)
            if (notCrossed(i))
                crossOutMultiplesOf(i);
    }

    private static int determineIterationLimit()
    {
        double iterationLimit = Math.sqrt(crossedOut.length);
        return (int) iterationLimit;
    }

    private static void crossOutMultiplesOf(int i)
    {
        for (int multiple = 2 * i; multiple < crossedOut.length; multiple += i)
            crossedOut[multiple] = true;
    }

    private static boolean notCrossed(int i)
    {
        return crossedOut[i] == false;
    }

    private static void putUncrossedIntegersIntoResult()
    {
        result = new int[numberOfUncrossedIntegers()];
        for (int j = 0, i = 2; i < crossedOut.length; i++)
            if (notCrossed(i))
                result[j++] = i;
    }

    private static int numberOfUncrossedIntegers()
    {
        int count = 0;
        for (int i = 2; i < crossedOut.length; i++)
            if (notCrossed(i))
                count++;
        return count;
    }
}
```

Fuente: Martin, R. C. (2008). Clean Code

5.2. Principios SOLID

Los principios SOLID son un conjunto de cinco principios de diseño orientado a objetos formulados por Robert C. Martin y agrupados bajo el acrónimo acuñado por Michael Feathers. Su objetivo es guiar al equipo hacia un diseño de código flexible, mantenible y resistente al cambio. Fueron desarrollados sistemáticamente por Martin (Martin, 2003) en Agile Software Development, Principles, Patterns, and Practices y retomados posteriormente en Clean Architecture (Martin, 2017).

1. S — Principio de Responsabilidad Única (Single Responsibility Principle)
   Una clase debe tener una única razón para cambiar. Esto significa que cada clase debe encapsular una única responsabilidad del sistema. Cuando una clase asume múltiples responsabilidades, un cambio en cualquiera de ellas puede afectar a las demás, aumentando el riesgo de introducir errores y dificultando el mantenimiento.
2. O — Principio Abierto/Cerrado (Open/Closed Principle)
   Una clase debe estar abierta para su extensión pero cerrada para su modificación. El comportamiento de una clase debe poder extenderse sin necesidad de modificar su código existente. Este principio fomenta el uso de abstracciones e interfaces como mecanismo para añadir nueva funcionalidad sin alterar el código que ya funciona y está probado.
3. L — Principio de Sustitución de Liskov (Liskov Substitution Principle)
   Una clase derivada debe poder sustituir a su clase base sin alterar el comportamiento del programa. Si una función espera un objeto de un tipo concreto, debe poder recibir cualquier subtipo de ese tipo sin que el resultado sea incorrecto. La violación de este principio es una señal de que la jerarquía de herencia está mal diseñada.
4. I — Principio de Segregación de Interfaces (Interface Segregation Principle)
   Una clase no debe verse obligada a implementar interfaces que no utiliza. Es preferible definir interfaces pequeñas y específicas que interfaces grandes y genéricas. Una interfaz con demasiados métodos obliga a las clases que la implementan a definir comportamientos que no les corresponden, generando dependencias innecesarias.
5. D — Principio de Inversión de Dependencias (Dependency Inversion Principle)
   Los módulos de alto nivel no deben depender de módulos de bajo nivel. Ambos deben depender de abstracciones. Este principio es el fundamento del diseño desacoplado:

en lugar de que una clase instancie directamente sus dependencias, estas se inyectan desde el exterior a través de interfaces. El resultado es un código más flexible, más testeable y más fácil de mantener.

Aplicar los principios SOLID de forma consistente no es una tarea puntual sino un hábito que el equipo debe cultivar en el día a día. Su valor se hace evidente a medio y largo plazo: un código diseñado con estos principios en mente es significativamente más fácil de extender, de probar y de mantener que un código que los ignora.

5.3. Refactoring

Refactoring: Improving the Design of Existing Code (Fowler, 1999) es la obra de referencia sobre la reestructuración del código existente. El propio Robert C. Martin hace referencia explícita a ella en el capítulo de síntomas y heurística de Clean Code, donde utiliza los code smells catalogados por Fowler como argumento para justificar cuándo y por qué el código necesita ser mejorado. Ambas obras son complementarias: Clean Code establece cómo debe ser el código para ser legible y mantenible, y Refactoring define cómo evolucionar el código para que tenga estas características.

5.3.1. Qué es el refactoring

En la obra se define el refactoring como el proceso de modificar la estructura interna del código sin alterar su comportamiento observable externo. No se trata de corregir errores ni de añadir funcionalidad, sino de mejorar la legibilidad, reducir la complejidad y eliminar la deuda técnica acumulada, dejando el sistema en mejores condiciones para afrontar los cambios futuros.

Esta diferencia es importante en el contexto del equipo: refactorizar no es reescribir código. Una reescritura implica cambiar el comportamiento del sistema; un refactoring mantiene el comportamiento. Contar con una batería de pruebas o tests facilita la tarea de refactoring, ya que disponemos de una garantía de que el código sigue funcionando igual después de la refactorización. Por este motivo Fowler insiste en que el refactoring y las pruebas son prácticas inseparables.

5.3.2. Code smells

Fowler introduce el concepto de code smell para describir las señales que indican que una pieza de código necesita ser refactorizado. No son errores en sentido estricto, ya que el código puede funcionar correctamente, pero son síntomas de un diseño que puede y debe mejorarse.

Los code smells más relevantes en el contexto del trabajo diario del equipo son los siguientes:

1. Código duplicado. La misma lógica aparece en varios lugares del sistema. Es el smell más frecuente y uno de los más costosos: cualquier modificación debe realizarse en múltiples puntos, multiplicando el riesgo de inconsistencias. La solución es extraer la lógica duplicada a una función o clase compartida, aplicando el principio DRY (Don't Repeat Yourself).
2. Función larga. Una función que hace demasiadas cosas y ha crecido sin control. Una función larga es una señal de que debe dividirse en funciones más pequeñas con responsabilidad única.
3. Clase grande. Una clase que ha acumulado demasiadas responsabilidades. Viola el principio de responsabilidad única y debe dividirse en clases más cohesionadas.
4. Lista de parámetros larga. Una función con demasiados parámetros es difícil de entender y de invocar correctamente. Fowler propone agrupar los parámetros relacionados en objetos de valor o introducir un objeto de parámetros.
5. Comentarios. Un comentario que explica qué hace el código es un síntoma de que el código no es suficientemente expresivo. La solución no es eliminar el comentario sino mejorar el código hasta que el comentario sea innecesario.
6. Envidia de funcionalidad. Un método que utiliza más datos o métodos de otra clase que de la suya propia. Es una señal de que ese método está en la clase equivocada y debería moverse.
7. Obsesión por los primitivos. Uso excesivo de tipos primitivos para representar conceptos del dominio. Un número de teléfono, un importe monetario o un código postal no son simples cadenas de texto: representan conceptos con sus propias reglas de validación y comportamiento, y merecen su propia clase.

5.3.3. Cuándo aplicar refactoring

Se identifican varios momentos naturales en el flujo de desarrollo en los que el refactoring debe aplicarse. El más importante es la regla del boy scout: deja el código mejor de como lo encontraste. Cada vez que un desarrollador trabaja sobre una parte del código tiene la responsabilidad de realizar pequeñas mejoras, sin necesidad de detener el desarrollo para dedicar un sprint completo a la refactorización.

Fowler propone además aplicar refactoring antes de añadir nueva funcionalidad, cuando el diseño existente dificulta la extensión del sistema. Refactorizar primero y añadir funcionalidad después es más eficiente que añadir funcionalidad sobre un diseño deficiente y acumular deuda técnica.

Por último, el refactoring es la respuesta natural a la detección de un code smell durante una revisión de código o una sesión de dimensionamiento. Cuando el equipo identifica un problema de diseño, debe abordarlo en el momento o registrarlo como una tarea técnica en el backlog para no perderlo.

6. Estructura de proyectos

El volumen de código que define una aplicación puede ser elevado, por ello la estructuración y nomenclatura de proyectos es vital. Definir una metodología para crear nuevos módulos facilita la implementación y lectura del código. Conocer en qué proyecto o solución se encuentra el código que se ejecuta al realizar un proceso de manera rápida e intuitiva es fundamental para mantener la agilidad del equipo a medida que el proyecto crece..

La nomenclatura de clases y métodos es fundamental para moverse ágilmente en un proyecto. Primamos la búsqueda intuitiva por nombre de clases y métodos en lugar de una búsqueda por explorador de ficheros. Un desarrollador que conoce las convenciones del equipo debe ser capaz de deducir el nombre de una clase que busca antes de buscarla, no después de encontrarla.

Nos apoyamos en el ejemplo implementado y publicado en el repositorio https://github.com/felipejorgeh/Delta.NearAcademy para detallar conceptos importantes.

El nombre del repositorio se corresponde con el nombre de una empresa y el nombre del producto: Delta y NearAcademy respectivamente.

6.1. Organización de proyectos

La nomenclatura de los proyectos sigue el patrón: {Empresa}.{Producto}.{Módulo}.{Capa}

Este patrón permite identificar de forma inmediata el contexto y la responsabilidad de cualquier proyecto con solo leer su nombre.

El módulo identifica el contexto de negocio al que pertenece el proyecto: Employee, Student, Core.

La capa identifica su responsabilidad técnica dentro del módulo: Domain, Business, Repository, Web, Injector o Test:

6.1.1. Domain

Contiene los objetos de transferencia de datos (DTOs) y las interfaces que definen las clases de los proyectos del módulo. Es el único proyecto que no tiene dependencias hacia ninguna otra capa: todos los demás proyectos del módulo dependen de él, nunca al revés. Esta regla

de dependencia garantiza que el dominio permanece independiente de cualquier detalle de infraestructura.

6.1.2. Business

Contiene la lógica de negocio del módulo. La lógica de negocio no conoce cómo se persisten los datos ni cómo se exponen al exterior: esos detalles están abstraídos por las interfaces definidas en Domain.

6.1.3. Repository

Contiene la implementación del acceso a datos. Implementa la interfaz de repositorio definida en Domain y encapsula todos los detalles de persistencia.

6.1.4. Web

Contiene el *controller* del módulo que expone los *endpoints* HTTP. Es la capa más externa del módulo y la única que interactúa con el framework web. Recibe las peticiones HTTP, las delega en la Application y devuelve la respuesta al cliente.

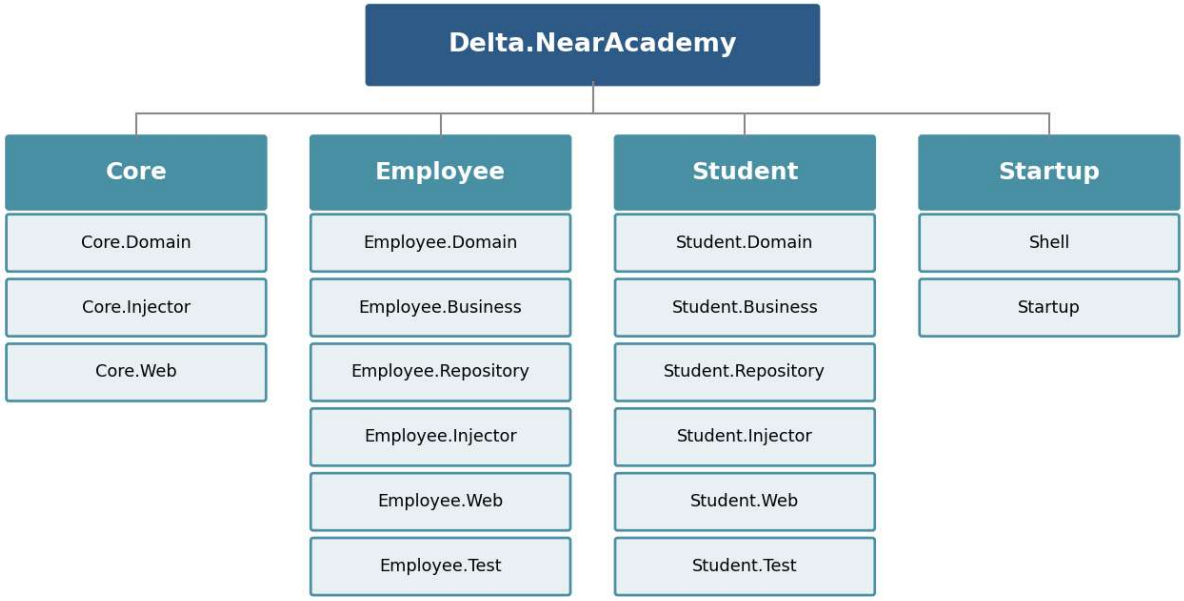
6.1.5. Injector

Es el proyecto responsable de registrar las dependencias del módulo en el contenedor de inyección de dependencias. Conecta las interfaces definidas en Domain con sus implementaciones en Business y Repository, centralizando en un único lugar toda la configuración de dependencias del módulo. Esto permite que el resto de capas permanezcan desacopladas.

6.1.6. Test

Contiene las pruebas unitarias del módulo. Se recomienda la implementación de clases Mock, clases que simulan el comportamiento de una interfaz, permitiéndonos testear una clase simulando el resto de las dependencias.

Figura 3 *Estructura de proyectos Delta.NearAcademy*



Fuente: Elaboración propia

6.2. Organización de clases e interfaces

Dentro de cada proyecto, la nomenclatura de las clases sigue igualmente una convención basada en el sufijo, que identifica el rol técnico de cada clase:

6.2.1. Application

Es la clase de entrada al módulo desde el exterior. Recibe las peticiones, las delega en el Service y devuelve el resultado. En el ejemplo, EmployeeApplication implementa IEmployeeApplication y es el punto de entrada que el controlador invoca. Su nombre sigue el patrón {Módulo}Application. Application ofrece versatilidad en el testeo y desacopla totalmente el Controller del Service.

6.2.2. Service

Contiene la lógica de negocio del módulo. Realiza las operaciones necesarias para completar un caso de uso, coordinando el acceso a datos a través del Repository y aplicando las reglas de negocio correspondientes. En el ejemplo, EmployeeService implementa IEmployeeService. Su nombre sigue el patrón {Módulo}Service.

6.2.3. Repository

Es la clase responsable del acceso a datos. Encapsula todos los detalles de persistencia, ya sea mediante Entity Framework u otro mecanismo de acceso a datos. En el ejemplo, EmployeeRepository implementa IEmployeeRepository. Su nombre sigue el patrón {Módulo}Repository.

6.2.4. Controller

Es la clase que expone los endpoints HTTP del módulo. Recibe las peticiones HTTP, las traduce en llamadas a la Application y devuelve la respuesta al cliente. En el ejemplo, EmployeeController delega en IEmployeeApplication. Su nombre sigue el patrón {Módulo}Controller.

Esta convención tiene una consecuencia directa en la navegación del código: dado el nombre de una clase o una interfaz, el desarrollador puede deducir en qué proyecto se encuentra sin necesidad de buscarlo. Una interfaz llamada IEmployeeRepository pertenece al proyecto Delta.NearAcademy.Employee.Domain; su implementación, EmployeeRepository, pertenece a Delta.NearAcademy.Employee.Repository.

El tipo de nombres de clase debe crecer, pero conviene que sean acordadas por convención dentro del equipo para mantener una implementación común. Algunos ejemplos habituales son: StudentMapper, para transformaciones de cambio de dominio, o StudentReporting, para lógica de generación de informes.

6.3. Módulo Core

El módulo Core contiene la definición de interfaces y clases utilizadas en el resto de los módulos. A diferencia de los módulos de negocio, el Core no representa un contexto de negocio concreto sino los mecanismos comunes que permiten ensamblar la aplicación y compartir lógica entre módulos. La naturaleza de estos objetos puede ser de dos tipos:

1. Objetos de arquitectura

En este apartado englobamos IInjector, Injector y ControllerAggregator.

IInjector es la interfaz que implementa la clase injector de cada módulo y que registra la resolución de interfaces. Cualquier nuevo módulo que se incorpore a la solución debe implementar esta interfaz para registrarse correctamente.

Injector es la clase que se encarga de registrar las dependencias de todos los módulos mediante reflexión.

ControllerAggregator es la clase que se encarga de registrar los controllers de todos los módulos mediante reflexión, siguiendo el mismo principio que el Injector. Cualquier controller que siga la convención de nomenclatura del equipo será descubierto y registrado automáticamente.

El uso de reflexión en ambas clases es una decisión de diseño deliberada: permite que la solución sea extensible sin modificar el Core ni el Startup. Añadir un nuevo módulo a la solución no requiere tocar ningún proyecto existente, sino únicamente implementar los contratos que el Core define.

2. Objetos de lógica común

Cuando existan Application, Service u otros objetos comunes a varios módulos, el módulo Core es su lugar natural. Los proyectos de cada módulo pueden referenciar su equivalente en Core, evitando así la duplicación de lógica compartida entre módulos.

Un ejemplo habitual son las clases de manejo de errores, los objetos de respuesta genéricos o las utilidades de transformación de datos que varios módulos necesitan. Centralizar estos objetos en Core garantiza que cualquier cambio en su implementación se propaga automáticamente a todos los módulos que los utilizan, sin necesidad de modificar cada módulo por separado.

6.4. Módulo Startup

El módulo Startup es el punto de entrada de la aplicación. Su responsabilidad es ensamblar todos los módulos de la solución y arrancar la aplicación. Está compuesto por dos proyectos: Shell y Startup.

Shell es un proyecto auxiliar cuyo cometido es arrastrar al output de compilación las referencias de proyectos que no llegan de forma indirecta a través de la cadena de dependencias de proyectos. En una arquitectura modular basada en reflexión, el Injector y el ControllerAggregator descubren los módulos en tiempo de ejecución, lo que significa que no existe una referencia explícita en tiempo de compilación hacia ellos. Sin Shell, los ensamblados de esos módulos pudieran no estar presentes en el directorio de salida y la reflexión no podría encontrarlos al arrancar la aplicación.

Startup contiene el Program.cs que es el responsable de inicializar la aplicación, registrar las dependencias de todos los módulos a través del Core.Injector y registrar los controladores a través del Core.Web. Incorpora además Swagger, herramienta que genera automáticamente la documentación de los endpoints HTTP de la API y proporciona una interfaz web para explorarlos e invocarlos durante el desarrollo. Swagger se convierte así en el punto de entrada natural para probar y documentar la API sin necesidad de herramientas externas.

7. Dimensionamiento y generación de tareas

El dimensionamiento es el proceso mediante el cual el equipo analiza una historia de usuario y la descompone en tareas de implementación concretas. Es uno de los momentos más importantes del proceso de desarrollo: una historia bien dimensionada garantiza que cualquier miembro del equipo puede abordar cualquier tarea con claridad y sin ambigüedad, independientemente de quién haya liderado el dimensionamiento.

Las reuniones de dimensionamiento ocurren principalmente en el Sprint Planning, pero pueden celebrarse también fuera de él cuando una historia de usuario requiere un análisis técnico previo más profundo. En ambos casos la dinámica es la misma: un miembro del equipo lidera la dimensión, expone su propuesta de implementación, el equipo debate y se generan las tareas resultantes.

7.1. Liderazgo

En las reuniones de dimensionamiento se trabaja la especificación, buscando el mejor encaje lógico en el proyecto, y la solución técnica. En un equipo Scrum conviene que exista más de una figura de desarrollador que pueda liderar los análisis funcionales y técnicos. Basado en la experiencia, este número de figuras tiende a ser tres, formando así una especie de triunvirato técnico. Es recomendable potenciar este triunvirato, la razón es práctica: dos personas se alinean con demasiada facilidad, ya sea por afinidad o por deferencia, lo que puede llevar a decisiones poco debatidas. Tres personas, en cambio, facilitan el debate, obligan a argumentar las propuestas y garantizan que cualquier decisión técnica relevante ha sido contrastada desde al menos tres perspectivas distintas. El acuerdo entre tres no es trivial, y esa dificultad es precisamente su valor: las decisiones que sobreviven al debate son decisiones sólidas.

Este liderazgo a tres no debe frenar o ser impedimento para que más desarrolladores puedan liderar determinadas dimensiones o para que participen activamente en los dimensionamientos

7.2. Dinámica de reunión

El líder de la dimensión presenta la historia de usuario al equipo. Antes de comenzar con la definición de tareas técnicas de desarrollo se debe trabajar en el encaje lógico de la solución.

Leer detenidamente la especificación y construir una propuesta lógica que se adecue al proyecto y si es necesario, analizar o crear *mockups* de interfaz gráfica. Si existe alguna duda o se plantea alguna problemática con la especificación y el desarrollo de una propuesta se debe consultar y resolver con el Product Owner.

Una vez la especificación está entendida y la propuesta de desarrollo trabajada, es entonces cuando podemos comenzar con la generación de tareas de desarrollo.

Las tareas de desarrollo deben ser detalladas, el objetivo es que finalizada la reunión todos los desarrolladores al asignarse una tarea de desarrollo para resolver no duden y sepan exactamente qué y cómo implementarlo.

Generadas las tareas, los desarrolladores valoran en tiempo estimado en horas lo que va a costar el desarrollo de cada tarea.

7.3. Ejemplo de reunión de dimensionamiento

A continuación, se ilustra el proceso de dimensionamiento con un ejemplo concreto basado en el proyecto de referencia Delta.NearAcademy.

La historia de usuario es la siguiente:

"Se requiere que se envíe un email de bienvenida al crear un nuevo empleado."

Se entiende lo requerido, al crear un nuevo trabajador se debe enviar un correo electrónico a la cuenta del trabajador. No se ha detallado ningún modelo de correo, así que se decide elaborar un correo básico con imagen de cabecera y un párrafo de bienvenida que contenga el nombre del empleado. Se propone crear una tarea para validar el formato con el Product Owner.

- Tarea 1: Configuración del servicio de email

Crear la interfaz IMailConfiguration en Delta.NearAcademy.Core.Domain y su implementación MailConfiguration en Delta.NearAcademy.Core.Business.

Método: MailConfig IMailConfiguration.GetConfig()

La implementación lee la configuración de envío de email del appsettings.json del proyecto Startup:

"MailConfiguration": {

"Host": "smtp.office365.com",

"Port": 587,

"EnableSsl": true,

"UserName": "noreply@nearacademy.com",

"Password": "...",

"FromAddress": "noreply@nearacademy.com",

"FromName": "NearAcademy"

}

- Tarea 2: Servicio de envío de email

Crear la interfaz IMailSender en Delta.NearAcademy.Core.Domain y su implementación MailSender en Delta.NearAcademy.Core.Business

Método: void IMailSender.Send(Mail mail)

Constructor: MailSender(IMailConfiguration mailConfiguration)

La clase Mail es un DTO que encapsula los datos del correo a enviar:

{

"To": "employee@nearacademy.com",

"Subject": "Bienvenido a NearAcademy",

"Body": "<h1>Bienvenido</h1> Tu cuenta ha sido creada. ",

"IsHtml": true

}

MailSender recibe IMailConfiguration por inyección de dependencias, obtiene la configuración SMTP mediante GetConfig() y establece la conexión con el servidor para enviar el correo mediante SmtpClient. Al residir en Core, cualquier módulo puede reutilizar este servicio sin duplicar la lógica de envío.

- Tarea 3: Envío de email de bienvenida al empleado

Crear la interfaz IEmployeeMailSender en Delta.NearAcademy.Employee.Domain y su implementación EmployeeMailSender en Delta.NearAcademy.Employee.Business

Método: void IEmployeeMailSender.SendWelcomeMail(Employee employee)

Constructor: EmployeeMailSender(IMailSender mailSender)

EmployeeMailSender recibe IMailSender por inyección de dependencias, construye el objeto Mail con los datos del empleado y delega el envío en IMailSender.Send

La lógica específica del email de bienvenida queda encapsulada en el módulo Employee, mientras que el mecanismo de envío permanece en Core, respetando el principio de responsabilidad única y evitando el acoplamiento entre módulos.

- Tarea 4: Registro de dependencias

Registrar en Delta.NearAcademy.Core.Injector las nuevas dependencias introducidas:

- IMailConfiguration → MailConfiguration

- IMailSender → MailSender

Registrar en Delta.NearAcademy.Employee.Injector la nueva dependencia introducida en el módulo Employee:

- IEmployeeMailSender → EmployeeMailSender

El registro de dependencias centralizado en los injectors de cada módulo garantiza que el contenedor de inyección de dependencias resuelve correctamente las interfaces en tiempo de ejecución, sin que ninguna otra capa del sistema necesite conocer las implementaciones concretas.

8. Pruebas

A lo largo de este capítulo se utilizan indistintamente los términos pruebas y tests para referirse al mismo concepto.

Las pruebas son una parte fundamental del desarrollo de software y una condición necesaria para la sostenibilidad del proyecto a largo plazo. Un código sin pruebas es un código difícil de modificar o refactorizar, porque cualquier cambio puede introducir errores difíciles de detectar.

En este capítulo distinguimos dos tipos de pruebas con naturaleza y propósito diferentes:

- Los tests de código son pruebas automatizadas escritas por los desarrolladores que verifican el comportamiento del sistema de forma programática, sin intervención humana.
- Los casos de prueba (*test cases*) son pruebas funcionales realizadas por una persona que sigue una lista de pasos predefinidos para validar el comportamiento del sistema desde la perspectiva del usuario.

Ambos tipos son complementarios: los tests de código verifican que el sistema funciona correctamente a nivel técnico, mientras que los casos de prueba verifican que el sistema se comporta correctamente desde el punto de vista del negocio y del usuario final.

Los tests nos aportan confianza y cierta seguridad para realizar modificaciones importantes en el código.

8.1. Tests de código

Los tests de código se ejecutan de forma automática y esta es una de las claves o puntos fuertes. Al realizar una modificación importante en el código, podemos ejecutar todos los tests y comprobar que su ejecución es correcta y todo sigue funcionando, no hemos roto ninguna funcionalidad.

Para tener seguridad y confiar en que nuestros tests cubren gran parte de las funcionalidades y código del sistema debemos determinar qué número de tests es el aconsejable y que testean.

El concepto de cobertura de código refleja esta situación: expresa el porcentaje de código cubierto por los tests.

Una aproximación para determinar un valor de cobertura adecuado puede establecerse mediante el principio de Pareto, regla empírica que establece que el 80% de los resultados proviene del 20% de las causas. Aplicado al testing, sugiere que testeando el 20% del código más crítico se previene el 80% de los errores posibles. Esta perspectiva orienta al equipo a identificar y priorizar las partes del código críticas, en lugar de perseguir una cobertura del 100%.

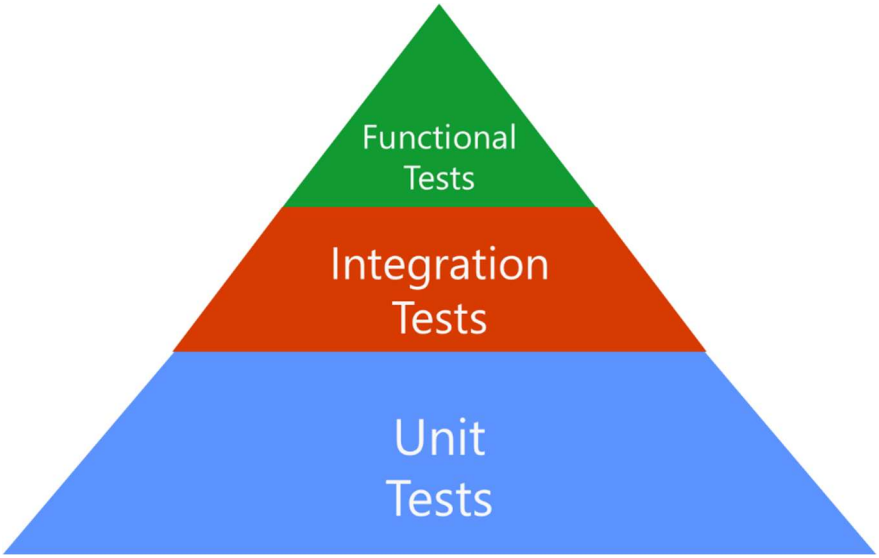
8.1.1. Tipos de tests

La pirámide de testing establece una jerarquía de tipos de pruebas que refleja tanto su volumen recomendado como su coste en tiempo y esfuerzo: cuanto más arriba en la pirámide, más costosa y lenta es la prueba.

- Tests unitarios verifican el comportamiento de una unidad de código de forma aislada, sin dependencias externas. Son los más abundantes, los más rápidos de ejecutar y los más baratos de mantener. Hacen uso de clases mock para simular las dependencias de la unidad bajo prueba, garantizando que el test verifica exclusivamente el comportamiento de esa unidad. En la estructura de proyectos propuesta, cada módulo dispone de su propio proyecto Test donde residen sus pruebas unitarias.
- Tests de integración verifican la interacción entre varios componentes del sistema: por ejemplo, la comunicación entre la capa de negocio y la base de datos, o entre dos módulos distintos. Son más lentos que los unitarios y requieren un entorno más cercano al real, pero verifican aspectos que los tests unitarios no pueden cubrir por su naturaleza aislada.
- Tests end-to-end verifican el comportamiento del sistema completo desde la perspectiva del usuario, simulando flujos reales de uso. Son los más costosos en tiempo de ejecución y mantenimiento, y deben reservarse para los flujos críticos del sistema.

La pirámide establece además una proporción recomendada: la base, formada por los tests unitarios, debe ser la más amplia; los tests de integración ocupan el nivel intermedio; y los tests end-to-end, en lo alto, deben ser los menos numerosos.

Figura 4 *Pirámide de testing*



Fuente: Microsoft Learn

8.1.2. Código testeable

Escribir tests no es solo una práctica de verificación sino un ejercicio de diseño. El código que es difícil de testear es, en la mayoría de los casos, código mal diseñado: con demasiadas responsabilidades, con dependencias acopladas o con efectos secundarios ocultos.

La arquitectura propuesta en Delta.NearAcademy, basada en interfaces e inyección de dependencias, facilita de forma natural la escritura de tests. Cualquier dependencia puede ser sustituida por un mock en el contexto de un test, lo que permite verificar el comportamiento de una clase de forma completamente aislada. Una clase que no puede testearse de forma aislada es una señal de que su diseño debe revisarse.

En el proyecto Delta.NearAcademy.Employee.Test encontramos un ejemplo de test unitario, CreateAndGetEmployeeTest. En el test sólo nos interesa testear el servicio EmployeeService y no el repositorio de acceso a datos, por eso se realiza un mock de IEmployeeRepository. En el test validamos que el trabajador se ha añadido correctamente, testeando la lógica de EmployeeService.

En la metodología de trabajo del equipo, cada historia de usuario tiene como mínimo una tarea de test de código para escribir los tests de código. En la reunión de dimensionamiento de la historia de usuario se debe crear la tarea.

Es buena práctica y en ocasiones muy recomendable definir los tests a implementar, detallando los nombres de los mismo. El nombre debe ser lo suficientemente descriptivo para no necesitar más detalle.

8.2. Casos de prueba y equipo de QA

Los tests de código automatizados no son suficientes por sí solos para garantizar la calidad del producto. Es necesario complementarlos con pruebas funcionales definidas por el equipo de desarrollo y ejecutadas por personas que validen el comportamiento del sistema desde la perspectiva del usuario. Esta responsabilidad recae sobre el equipo de QA.

QA son las siglas de Quality Assurance (Aseguramiento de la Calidad), el equipo de QA es el responsable de verificar que el producto cumple los requisitos funcionales y de calidad establecidos

El equipo de QA está presente en las reuniones de refinamiento de Backlog, de esta forma garantizamos que participe en el debate de definición de la solución. También acude a la parte inicial de la reunión de sprint planning dónde se presentan las funcionalidades a abordar en el sprint, de esta forma se prepara para disponer de todo lo necesario para ejecutar los casos de prueba.

El equipo de desarrollo es el encargado de escribir los casos de prueba, ejecutarlos e indicar al equipo QA que los ejecute. Este proceso queda definido para toda funcionalidad, en la reunión de dimensión se generan tres tareas con nombre: Test for QA, Test by devs y Test by QA. Los casos de prueba se escriben en una herramienta adecuada para tal efecto como puede ser Azure Test Plans. Todas las tareas de tests se deben ejecutar en un entorno limpio, un entorno donde se despliega la aplicación.

La tarea de Test for QA la escribe un desarrollador y en ella se detallan las acciones y su resultado. Un ejemplo de casos de prueba para el proyecto Delta.NearAcademy:

1. **Acción**: Acceder a la URL de Swagger de la aplicación
   Resultado: Se muestra la interfaz de Swagger con los endpoints disponibles

2. **Acción**: Localizar el endpoint POST /api/employee, expandirlo y pulsar Try it out
   Resultado: El endpoint queda habilitado para su ejecución

3. **Acción**: Introducir en el body el siguiente JSON y pulsar *Execute*: { "name": "John Doe", "email": "john.doe@nearacademy.com" }
   Resultado: La API devuelve un código HTTP 200 y el identificador del empleado creado
4. **Acción**: Localizar el endpoint GET /api/employee/{id}, expandirlo y pulsar Try it out
   Resultado: El endpoint queda habilitado para su ejecución

5. **Acción**: Introducir el identificador obtenido en el paso 3 y pulsar Execute
   Resultado: La API devuelve un código HTTP 200 y los datos del empleado: { "id": "{id}", "name": "John Doe", "email": "john.doe@nearacademy.com" }

El propio desarrollador que ha creado el test debe ejecutarlo.

Una vez concluida la tarea Test for QA se puede completar la tarea Test by devs, esta tarea la ejecuta otro desarrollador distinto al que realizó la tarea Test for QA. Al ejecutar de nuevo el test, otro desarrollador valida el test escrito y su ejecución.

Finalmente, concluida la tarea de Test by devs, se asigna la tarea Test by QA al equipo de QA. Concluida la tarea Test by QA se cierra la historia de usuario.

Si en algún momento de la ejecución de estas tareas de test se detecta un problema se generan tareas para solucionarlos, parando la tarea de test correspondiente y retomándola una vez corregido los problemas.

9. Flujo de trabajo con Git

Git es el sistema de control de código fuente estándar en el mundo del desarrollo de software. Entre sus principales ventajas destacan la capacidad de trabajar de forma distribuida sin necesidad de conexión a un servidor central, el seguimiento completo del historial de cambios del código, la posibilidad de trabajar en ramas independientes para aislar desarrollos, la facilidad para revertir cambios y la integración con la mayoría de las herramientas y plataformas de desarrollo existentes. No es objetivo de este punto profundizar en el manejo de Git, se asume que el equipo de desarrollo conoce su uso básico. El objetivo es definir una estrategia de trabajo con Git que el equipo pueda aplicar de forma consistente y coordinada.

La estrategia más básica de trabajo es una estrategia basada en una única rama identificada como trunk. Los commits se realizan directamente sobre esta rama, denominada master o main. Normalmente, master debe estar disponible para desplegar a final de sprint. Si los desarrollos son complejos o algún desarrollo se alarga, esta estrategia puede presentar bloqueos: un desarrollo incompleto en master impide el despliegue del resto del trabajo completado.

Por ello escogemos un flujo de trabajo basado en ramas feature y ramas task. Este enfoque permite aislar cada desarrollo en su propia rama, de modo que el trabajo en curso no afecta a la estabilidad de master y el equipo puede trabajar en paralelo sin interferencias.

9.1. Ramas feature y task

En la estrategia de trabajo propuesta, cualquier desarrollo tiene su origen en una historia de usuario, que a su vez contiene las tareas de desarrollo y las tareas de test generadas en la reunión de dimensionamiento. Este modelo de trabajo tiene su reflejo directo en la estrategia de ramas.

Para cada historia de usuario se crea una rama feature desde el último commit de *master*, con la siguiente nomenclatura: Feature/{id y nombre de la historia de usuario}

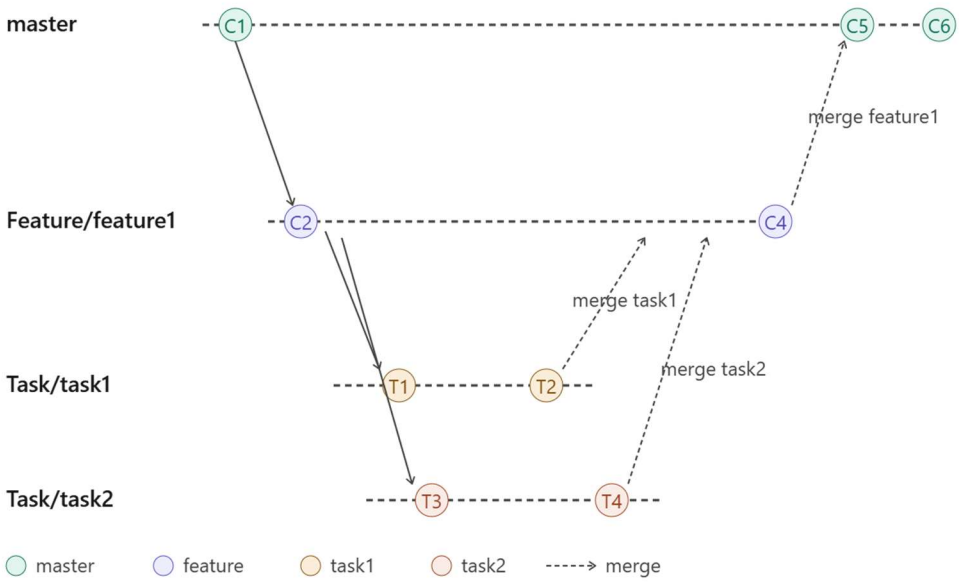
Para cada tarea de desarrollo se crea una rama task desde el último commit de la rama feature de su historia de usuario, con la siguiente nomenclatura: Task/{id y nombre de la tarea}

Al finalizar el desarrollo de una tarea, se realiza el merge de la rama task sobre la rama feature correspondiente. Una vez completadas todas las tareas de la historia de usuario, siendo la última la tarea *Test by QA*, se realiza el merge de la rama feature sobre *master*.

Este flujo de trabajo implica la creación de tres tareas adicionales en la historia de usuario, que deben generarse durante el dimensionamiento: Crear rama feature, Realizar merge de feature en master y Eliminar rama feature

Las ramas task no requieren eliminación manual: como veremos en el capítulo siguiente, el proceso de CI/CD se encarga de eliminarlas automáticamente al completar el merge.

Figura 5 *Ramas feature y task*



Fuente: Elaboración propia

9.2. Ramas release y hotfix

En una aplicación web alojada en la nube, el código de master es desplegado en producción y todos los clientes acceden al sitio web ejecutando siempre el último código. En esta situación, cuando se detecta un error y se corrige en master, la corrección llega a producción simplemente desplegando master.

Sin embargo, existen escenarios donde esta situación varía: aplicaciones web desplegadas on premise (en un servidor propio del cliente) o aplicaciones de escritorio instaladas en máquinas

del cliente. En estos casos, desplegar master completo puede no ser la mejor opción por sus implicaciones, y se requiere realizar un parche o patch que contenga únicamente las modificaciones necesarias.

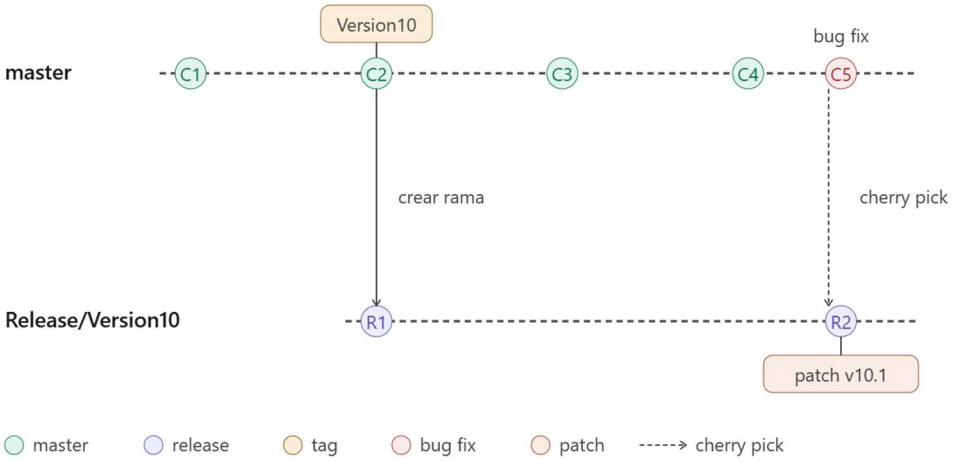
Para gestionar este escenario empleamos ramas release. En el commit de master que tiene desplegado el cliente se genera un tag, concepto de Git que permite marcar un punto concreto del historial del código. Este tag se identifica habitualmente con el nombre de versión del producto, por ejemplo, Version10. Además del tag, se genera una rama desde ese mismo commit con la nomenclatura: Release/Version10

Cuando se detecta un bug en esa versión, se corrige en master y el commit de corrección se lleva a la rama release mediante cherry pick, operación de Git que permite aplicar un commit concreto sobre otra rama sin necesidad de fusionar el historial completo.

Una vez aplicada la corrección en la rama release, se genera el parche a partir de ella. El parche representa únicamente las librerías modificadas por el commit de corrección y se despliega en la instalación del cliente, evitando así la necesidad de realizar un despliegue completo.

Una corrección o bug se detalla en una historia de usuario, si esta corrección a realizar sobre master es necesario que se lleve a un rama release, crearemos tareas para este propósito después de la tarea de Test by QA: Cherry pick a Release/Version10, Test for QA Release/Version10, Test by devs Release/Version10 y Test by QA Release/Version10.

Figura 6 *Ramas release*



Fuente: Elaboración propia

10. Integración continua y entrega continua (CI/CD)

La integración continua y la entrega continua, conocidas por las siglas CI/CD (*Continuous Integration / Continuous Delivery*), son un conjunto de prácticas y mecanismos automatizados que garantizan la calidad del código y la disponibilidad del producto en todo momento.

La integración continua consiste en ejecutar de forma automática un conjunto de validaciones cada vez que un desarrollador integra nuevo código en el repositorio: compilación, ejecución de tests y análisis estático del código. Su objetivo es detectar errores de forma inmediata, en el momento en que se introduce el cambio, evitando que los problemas se acumulen y sean más costosos de resolver.

La entrega continua extiende la integración continua y permite que el código de la rama principal se despliegue automáticamente cuando se requiera.

Nuestro repositorio Git se encuentra alojado en una plataforma, en la nube o en local, que nos permite realizar acciones sobre el código de forma automática, estas acciones las denominamos pipeline y nos permiten gestionar los procesos de CI/CD.

10.1. Integración continua (CI)

Al realizar un commit en el repositorio y realizar push sobre el repositorio remoto podemos lanzar una compilación de todos los proyectos y verificar que su compilación es correcta. Para ello podemos generar un pipeline indicando que cualquier integración de commits en las ramas master, feature o release desencadene una compilación de los proyectos.

Ejemplo en CiSources.yml en Delta.NearAcademy.

Excluimos las ramas task de esta validación porque una tarea de desarrollo la realiza un único desarrollador, la validación se puede realizar en local y no tiene repercusión hasta llegar a la rama feature.

La ejecución del pipeline CiSources.yml es más valiosa si lo ejecutamos antes de realizar el merge y si el resultado no es satisfactorio decidimos no integrar los cambios en el repositorio, esta acción define la esencia de una Pull request.

10.1.1. Pull request

En la plataforma podemos configurar que sobre las ramas master, feature y release no se pueda realizar la acción de push, acción de git para integrar commits. Sólo podemos hacer push sobre ramas task. Mediante este mecanismo conseguimos que para introducir cambios en esas ramas protegidas debamos hacer una petición a la plataforma. Esta petición se denomina Pull request y nos permite hacer un merge de una rama origen a una rama destino.

Por ello la manera de integrar cambios en una rama master, feature y release es realizar un merge de una rama task a una de estas ramas.

En la pull request podemos añadir comprobaciones:

- Antes de realizar el merge, simular el merge de las ramas y ejecutar el pipeline CiSources.yaml. Garantizando que los cambios a integrar no rompen código.
- Exigir revisiones del código por parte de otros desarrolladores. La plataforma necesita que más desarrolladores consulten la Pull request y la validen.

10.2. Entrega continua (CD)

La entrega continua automatiza el proceso de despliegue del software, garantizando que el código puede llegar al entorno de producción de forma controlada y reproducible.

El pipeline de entrega continua se ejecuta una vez que el pipeline de integración continua ha finalizado correctamente: compilación superada y tests ejecutados. Sólo cuando todas estas validaciones son correctas se desencadena el proceso de despliegue. Esta secuencia garantiza que únicamente el código validado llega al entorno de producción.

El despliegue puede adoptar formas distintas según la estrategia de versionado.

En una aplicación web sin versionado explícito, el pipeline compila los proyectos, genera los ensamblados resultantes y los despliega directamente en el servidor web.

En un proyecto con versionado, el pipeline genera además un artefacto: un fichero comprimido o instalador que contiene exactamente el código desplegado, identificado con el número de versión correspondiente, por ejemplo NearAcademy_v10.1.zip. Este artefacto queda almacenado en la plataforma y puede ser descargado y desplegado en cualquier momento, garantizando la trazabilidad completa entre el código del repositorio y el software instalado en cada cliente.

Al ejecutarse el despliegue, el pipeline genera automáticamente el tag de versión correspondiente en el repositorio y, si el proyecto lo requiere, la rama release asociada. De este modo el versionado queda integrado en el flujo de trabajo y no depende de una acción manual que pueda omitirse u olvidarse.

11. Plataforma DevOps

El término DevOps surge de la unión de Development (desarrollo) y Operations (operaciones), y representa una cultura de trabajo que busca eliminar la separación tradicional entre los equipos de desarrollo de software y los equipos de operaciones responsables de su despliegue y mantenimiento. Su objetivo es acortar el ciclo de vida del desarrollo, aumentar la frecuencia de las entregas y mejorar la calidad del producto mediante la automatización y la colaboración continua entre ambas disciplinas.

En la práctica, DevOps se materializa en un conjunto de herramientas y procesos que automatizan las tareas repetitivas del ciclo de desarrollo: integración del código, ejecución de tests, análisis de calidad y despliegue. Las plataformas DevOps centralizan estas capacidades en un único entorno, ofreciendo alojamiento del repositorio de código, gestión del trabajo, pipelines de CI/CD y mecanismos de revisión de código como las pull requests. Además, estas plataformas suelen incorporar herramientas para la gestión del proceso Scrum y para la definición y ejecución de casos de prueba, cubriendo así la mayor parte de las necesidades del equipo de desarrollo en un único entorno integrado.

Existen diferentes plataformas de DevOps disponibles en el mercado, debemos escoger una herramienta para gestionar los siguientes apartados detallados en este trabajo:

- 7. Dimensionamiento y generación de tareas
- 8. Pruebas
- 9. Flujo de trabajo con Git
- 10. Integración continua y entrega continua (CI/CD)

Para un ecosistema Microsoft, el proyecto Delta.NearAcademy está escrito en .NET, recomendamos Azure Devops.

Azure DevOps es la plataforma DevOps desarrollada por Microsoft. Organiza sus capacidades en módulos independientes, cada uno de los cuales da soporte a uno de los apartados desarrollados en este trabajo:

- Azure Boards da soporte a la gestión SCRUM: backlog, sprints, historias de usuario, tareas y tablero kanban.

- Azure Repos da soporte al flujo de trabajo con Git: repositorio, ramas feature, task y release, y pull requests.
- Azure Pipelines da soporte a la integración y entrega continua: pipelines de CI/CD definidos mediante ficheros YAML.
- Azure Test Plans da soporte a los casos de prueba: planes, suites y casos de prueba con sus pasos y resultados esperados.

La principal ventaja de esta plataforma es la integración de todos sus módulos en un único entorno. Una historia de usuario gestionada en Azure Boards se vincula con las ramas de Azure Repos donde se implementa, con los pipelines de Azure Pipelines que la validan y con los casos de prueba de Azure Test Plans que verifican su funcionamiento, proporcionando una trazabilidad completa desde la planificación hasta el despliegue.

12. Scrum de Scrums

A medida que un producto crece en complejidad y el número de desarrolladores aumenta, un único equipo Scrum puede resultar insuficiente para abordar el volumen de trabajo requerido. En esta situación es necesario formar múltiples equipos que trabajen de forma coordinada sobre un mismo producto. Sin embargo, escalar el número de equipos sin un marco de coordinación adecuado genera los mismos problemas que escalar el número de desarrolladores sin un marco de trabajo: desacoplamiento, duplicación de esfuerzos, dependencias no gestionadas y pérdida de visión global del producto.

Scrum at Scale (Sutherland, 2021) es el marco formal para abordar este reto. Su objetivo es aplicar los principios de Scrum a organizaciones con múltiples equipos, manteniendo la agilidad y la capacidad de inspección y adaptación que caracterizan a un único equipo Scrum bien estructurado.

12.1. Principios de Scrum at Scale

Scrum at Scale parte de un principio fundamental: escalar Scrum no significa crear una burocracia más grande, sino replicar la estructura de un equipo Scrum a un nivel superior. Del mismo modo que un equipo Scrum tiene sus roles, ceremonias y artefactos, Scrum at Scale define una capa de coordinación entre equipos que sigue los mismos principios.

12.2. Scrum of Scrums

El mecanismo central de Scrum at Scale es el Scrum of Scrums (SoS). Un Scrum of Scrums agrupa varios equipos Scrum y funciona como un equipo Scrum en sí mismo: tiene sus propios roles, sus propios eventos y sus propias responsabilidades. Un representante de cada equipo, habitualmente el Scrum Master o un líder técnico, forma parte del Scrum of Scrums y actúa como enlace entre su equipo y el resto.
Dado que el Scrum of Scrums funciona como un equipo Scrum, necesita celebrar versiones escaladas de todos los eventos de Scrum.
El Scrum Master del Scrum de Scrums se llama Scrum of Scrums Master (SoSM). El Scrum of Scrums Master es responsable de garantizar que los eventos de Scrum of Scrums se celebren.

12.3. Meta Scrum

A nivel de producto, Scrum at Scale introduce el concepto de Meta Scrum: una reunión periódica en la que los Product Owners de todos los equipos se coordinan para gestionar y priorizar el backlog común del producto. El backlog es único para toda la organización: no existe un backlog por equipo sino una única lista ordenada de historias de usuario que refleja las prioridades del producto en su conjunto. El Meta Scrum garantiza que la priorización del backlog común es coherente con los objetivos de negocio y que cada equipo trabaja en aquello que aporta mayor valor al producto, independientemente de qué equipo lo construya.

12.4. Aplicación práctica de Scrum at Scale

Si un equipo Scrum se aproxima a 9 miembros, el equipo empieza a dejar de ser eficiente. Una de las principales ineficiencias es la carga cognitiva que supone gestionar un sprint con un volumen elevado de historias de usuario. Si entran diez historias de usuario en un sprint, cada miembro del equipo debe conocer y comprender el detalle de todas ellas: su implementación, sus dependencias y su estado en cada momento. La carga cognitiva crece de forma proporcional al número de miembros y al volumen de historias, llegando a un punto en el que ningún desarrollador puede mantener una visión completa del sprint sin un esfuerzo desproporcionado.

Una opción de crecimiento es crear más de un equipo Scrum siguiendo las recomendaciones de Scrum at Scale, pero esa transición puede ser complicada y plantea retos importantes, uno de ellos salvaguardar un código alineado con la arquitectura y que cumpla con las características de buenas prácticas comentadas en este trabajo.

Una forma de comenzar el desacoplamiento puede ser la implementación de Scrum of Scrums de forma parcial.

Disponemos de un equipo que partimos en dos equipos de tamaño similar. Permitimos que estos nuevos equipos, A y B, puedan crecer de forma independiente, aumentando la capacidad de desarrollo.

Todos los miembros de ambos equipos acuden a las reuniones detalladas en el apartado 4.5:

- Refinamiento del Backlog
- Pre Sprint Planning

- Next Sprint
- Sprint Planning
- Daily
- Pre Sprint Review
- Sprint Review
- Retrospectiva

Modificamos el Sprint Planning subdividiéndolo en:

- Sprint Planning intro

  Acuden los miembros de los dos equipos y se enumeran las funcionalidades a desarrollar en el sprint
- Sprint Planning

  Reunión en la que acuden los miembros de un equipo y donde tienen lugar las dimensiones. Existen dos reuniones de Sprint Planning, Sprint Planning A y Sprint Planning B

Además, se añade una reunión cada dos o tres días en el sprint denominada Scrum of Scrums Daily, en la que los Scrum Masters se alinean. La mecánica es idéntica a una reunión Daily.

Finalmente, con el objetivo de compartir el código implementado entre los dos equipos se celebra una reunión denominada Sprint Sync el penúltimo día del sprint.

13. Evaluación de resultados basada en experiencia profesional

La metodología propuesta en este trabajo no es el resultado de una elaboración puramente teórica, sino el estudio y evolución de veinte años de experiencia práctica en dos empresas y dos proyectos de naturaleza completamente distinta.

La primera experiencia tuvo lugar en una empresa con una aplicación web de control de actividades empresariales, donde se produjo la primera toma de contacto con la metodología SCRUM y los conceptos sobre implementación de código descritos principalmente en *Clean Code* (Martin, 2008) y *Clean Architecture* (Martin, 2017).

Posteriormente en una empresa con un sistema cliente-servidor de control de accesos, con cliente de escritorio y servidor con servicios web.

La experiencia más ilustrativa de la implantación de esta metodología se produjo en la segunda empresa, el sistema de control de accesos, donde me incorporé a un equipo que presentaba todas las características descritas en la motivación de este trabajo: ausencia de cultura de equipo, desarrolladores trabajando de forma completamente independiente bajo criterios individuales y sin ningún proceso coordinado, código espagueti y sin convenciones.

La manifestación más inmediata de este desorden era la imposibilidad de garantizar un despliegue fiable del sistema: cada desarrollador probaba en su propia máquina, pero el software no funcionaba de forma consistente en otros entornos. Existían innumerables bugs, se corregía un detalle y se rompía otro; no existían tests de ningún tipo. La implantación de la metodología se realizó de forma gradual, respetando la capacidad de absorción del equipo y priorizando los cambios de mayor impacto inmediato.

El primer paso fue tomar el control del código y la arquitectura. Comprender y organizar mínimamente el código para poder dar salida a nuevos desarrollos y comenzar a refactorizar código para apuntalar la arquitectura del proyecto. Es esencial en este primer punto ser o disponer de un desarrollador senior que ofrezca resultados a corto plazo y transmita confianza y seguridad.

Una vez consolidada la base técnica, se introdujo el marco de trabajo SCRUM junto con la metodología de pruebas. La introducción de las ceremonias, los roles y el proceso de dimensionamiento transformó la forma en que el equipo planificaba y coordinaba su

trabajo. El dimensionamiento introduce un concepto clave, controlar el código que va a escribir el equipo antes de ser escrito. En mi opinión es el punto clave de toda la metodología, sin un código bien escrito y mantenible nada se sostiene en el tiempo.

La incorporación de los tests de código y los casos de prueba dotó al equipo de la confianza necesaria para modificar y extender el sistema sin temor a introducir errores no detectados.

En una tercera fase se implantó la estrategia de ramas Git y el pipeline de integración continua con pull requests. Este paso automatizó las validaciones de calidad y eliminó definitivamente el problema de los entornos inconsistentes que había caracterizado la situación inicial.

Los resultados observados tras la implantación completa de la metodología fueron significativos en tres dimensiones. La reducción de errores en producción fue notable, al existir una red de pruebas automáticas y un proceso de validación sistemático antes de cualquier integración de código. Los tiempos de entrega mejoraron gracias a la planificación estructurada mediante sprints y al dimensionamiento previo de cada historia de usuario. La cohesión del equipo y el conocimiento compartido del sistema aumentaron de forma sostenida, eliminando las dependencias críticas de personas individuales que habían caracterizado la situación inicial. El código es mantenible, las refactorizaciones son abordables y se facilita la evolución tecnológica.

Esta experiencia práctica confirma la validez y la utilidad de la metodología propuesta.

14. Conclusiones y trabajo futuro

Este trabajo partió de un problema habitual y complejo en el desarrollo de software profesional: la ausencia de un verdadero equipo de desarrollo, con desarrolladores trabajando de forma aislada, sin una cultura técnica compartida y sin un marco de trabajo que coordine su actividad. Ante este problema, el objetivo ha sido diseñar un marco de trabajo mínimo que integre metodologías ágiles, prácticas técnicas de calidad de código y herramientas de gestión, capaz de dotar a cualquier equipo de los mecanismos necesarios para organizarse, coordinarse y entregar software de forma sostenible y mantenible a lo largo del tiempo.

A lo largo de los capítulos anteriores se ha construido este marco de forma progresiva, partiendo de la adaptación de SCRUM como base metodológica, profundizando en las prácticas de calidad del código, definiendo una estructura de proyectos coherente, detallando el proceso de dimensionamiento, integrando las pruebas, estableciendo una estrategia de gestión del código fuente con Git y CI/CD, y finalmente abordando el escalado hacia múltiples equipos. En este capítulo se valora en qué medida se han alcanzado los objetivos planteados al inicio del trabajo y se señalan las líneas de trabajo futuro que abre esta contribución.

14.1. Conclusiones del trabajo

El objetivo general del trabajo, diseñar un marco de trabajo para equipos de desarrollo de software, se considera alcanzado. El marco propuesto cubre el ciclo completo de trabajo de un equipo, desde la planificación hasta el despliegue. A continuación se valora el grado de consecución de cada uno de los objetivos específicos planteados.

El primer objetivo, utilizar una metodología ágil como base del marco de trabajo, se ha alcanzado mediante la adopción de SCRUM, adaptado con los matices necesarios para ajustarse a la realidad de un equipo concreto. Se han definido los roles, las ceremonias y los artefactos, incorporando elementos propios como la figura del líder técnico y ceremonias adicionales como el Pre Sprint Planning y el Next Sprint.

El segundo objetivo, proponer criterios y buenas prácticas de implementación, se ha alcanzado mediante el desarrollo de los principios de Clean Code, los principios SOLID (Martin, 2003) y la práctica del refactoring, estableciendo una metodología de codificación a la que el equipo puede recurrir como argumento técnico en las discusiones de diseño.

El tercer objetivo, definir una estructura organizativa que fomente el conocimiento compartido, se ha alcanzado tanto a nivel de equipo, con la propiedad colectiva del código y la eliminación de dependencias críticas de personas, como a nivel de código, con la estructura de proyectos basada en Clean Architecture y las convenciones de nomenclatura que permiten a cualquier desarrollador orientarse en el sistema.

El cuarto objetivo, establecer un proceso de planificación que detalle cada tarea antes de su implementación, se ha alcanzado mediante el proceso de dimensionamiento, que garantiza que antes de escribir una sola línea de código existe una visión técnica compartida de cómo abordar cada historia de usuario.

El quinto objetivo, definir una estrategia de gestión del código fuente, se ha alcanzado mediante el flujo de trabajo con Git basado en ramas feature, task y release, complementado con las políticas de pull request que protegen las ramas principales.

El sexto objetivo, integrar un sistema de pruebas apoyado en CI/CD, se ha alcanzado distinguiendo entre los tests de código automatizados y los casos de prueba funcionales del equipo de QA, e integrando ambos en el pipeline de integración y entrega continua.

El séptimo objetivo, explorar la escalabilidad del marco, se ha alcanzado mediante la introducción de Scrum at Scale y la descripción de una estrategia práctica de transición desde un único equipo hacia múltiples equipos coordinados.

En conjunto, la consecución de todos los objetivos específicos permite afirmar que el objetivo general ha sido alcanzado: el trabajo proporciona un marco completo, coherente y aplicable en la práctica para la gestión de un equipo de desarrollo de software.

14.2. Líneas de trabajo futuro

El marco de trabajo propuesto abre diversas líneas de continuación que podrían ampliar y enriquecer la contribución de este trabajo.

Una primera línea es la profundización en el escalado a múltiples equipos. En este trabajo se ha introducido Scrum at Scale y se ha descrito una estrategia práctica de transición, pero el comportamiento del marco en organizaciones con un número elevado de equipos coordinados requeriría un estudio más extenso, idealmente validado con la experiencia de implantación en un entorno multi-equipo real y maduro.

Una segunda línea es la incorporación de métricas objetivas que permitan cuantificar el impacto del marco de trabajo. Aunque en la evaluación de resultados basada en experiencia profesional se han descrito mejoras observadas en la reducción de errores, los tiempos de entrega y la cohesión del equipo, sería valioso definir un conjunto de métricas concretas, como la velocidad del equipo, la cobertura de código, el tiempo medio de resolución de incidencias o la frecuencia de despliegue, que permitan medir de forma sistemática la evolución del equipo y el efecto de cada práctica introducida.

Una tercera línea es la incorporación de la gestión del versionado de la base de datos al marco de trabajo. Los cambios en el esquema de datos presentan retos específicos de control de versiones, despliegue y reversión que no se han abordado en profundidad en este trabajo, y su integración en la estrategia de Git y en el pipeline de CI/CD constituye una extensión natural del marco propuesto.

Referencias bibliográficas

Anderson, D. J. (2010). *Kanban: Successful evolutionary change for your technology business*. Blue Hole Press.

Beck, K. (1999). Extreme programming explained: Embrace change. Addison-Wesley.

Beck, K. (2002). Test-driven development: By example. Addison-Wesley.

Beck, K., Beedle, M., van Bennekum, A., Cockburn, A., Cunningham, W., Fowler, M., Grenning, J., Highsmith, J., Hunt, A., Jeffries, R., Kern, J., Marick, B., Martin, R. C., Mellor, S., Schwaber, K., Sutherland, J., & Thomas, D. (2001). Manifesto for agile software development. https://agilemanifesto.org

Chacon, S., & Straub, B. (2014). Pro Git (2nd ed.). Apress. https://git-scm.com/book

Driessen, V. (2010). A successful Git branching model. https://nvie.com/posts/a-successful-git-branching-model

Fowler, M. (1999). Refactoring: Improving the design of existing code. Addison-Wesley.

Fowler, M. (2006). Continuous integration. https://martinfowler.com/articles/continuousIntegration.html

Fowler, M. (2012). TestPyramid. https://martinfowler.com/bliki/TestPyramid.html

Fowler, M. (2018). Refactoring: Improving the design of existing code (2nd ed.). Addison-Wesley.

Hammant, P. (2020). Trunk based development. https://trunkbaseddevelopment.com

Humble, J., & Farley, D. (2010). Continuous delivery: Reliable software releases through build, test, and deployment automation. Addison-Wesley.

Kruchten, P. (2003). The rational unified process: An introduction (3rd ed.). Addison-Wesley.

Martin, R. C. (2003). Agile software development, principles, patterns, and practices. Prentice Hall.

Martin, R. C. (2008). Clean code: A handbook of agile software craftsmanship. Prentice Hall.

Martin, R. C. (2017). Clean architecture: A craftsman's guide to software structure and design. Prentice Hall.

Royce, W. W. (1970). Managing the development of large software systems. Proceedings of IEEE WESCON, 26, 1–9.

Schwaber, K., & Sutherland, J. (2020). The Scrum guide. https://scrumguides.org

Skelton, M., & Pais, M. (2019). Team topologies: Organizing business and technology teams for fast flow. IT Revolution Press.

Smart, J. F. (2014). BDD in action: Behavior-driven development for the whole software lifecycle. Manning Publications.

Sutherland, J. (2021). Scrum at scale guide. https://www.scrumatscale.com/scrum-at-scale-guide